

Bases de Datos

Clase 11

Fundamentos de EDD y Almacenamiento e Índices

Contenidos revisados

Hasta ahora se ha estudiado

- El modelo relacional
- Consultas en álgebra relacional
- Modelamiento de un problema con el modelo E/R
- Transformación de un MER a un esquema relacional
- Dependencias funcionales y anomalías
- Formas normales 1, 2, 3 y BC

  Proyecto E1
- SQL

  Proyecto E2
- Vistas, triggers y stored procedures
- Transacciones, ACID, schedules y locks
- Recuperación ante fallas

Clase de hoy: Índices

Latencia de acceso a los datos

"Acceder a RAM es como tomar algo en tu escritorio; acceder a SSD es como caminar desde la biblioteca al edificio Raul Deves, acceder a disco HDD es como ir en bus desde Santiago a Arica."

En general, la operación más lenta de un DBMS es la lectura de datos desde disco; por ello, el objetivo es minimizar el número de operaciones de I/O

Modelo de costos

Memoria Principal vs. Memoria Secundaria



Se cuentan los accesos (lectura/escritura) a memoria secundaria

Memoria Secundaria

- Los datos se almacenan en memoria secundaria
- La lectura se realiza por **páginas**
- Una página tiene un tamaño de *B* tuplas



Los accesos a memoria principal son despreciables

Memoria Principal

- Los datos son llevados a memoria principal
- La memoria tiene una capacidad de *M* páginas

Modelo de costos

Memoria Secundaria

Una página tiene un tamaño de *B* tuplas

Memoria Principal

La memoria tiene una capacidad de *M* páginas

¿Cuánto cuesta leer n tuplas de memoria secundaria?

$$\left\lceil \frac{n}{B} \right\rceil$$

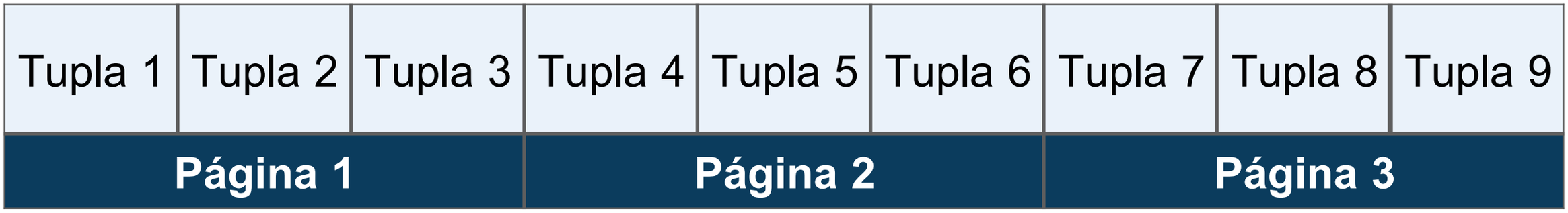
¿Cuántas páginas usa una relación R?

$$\left\lceil \frac{|R|}{B} \right\rceil$$

Almacenamiento de una tabla

Considérese una tabla T(a int, b text) con 9 tuplas

Disco Duro



Costo I/O (Input/Output)



El costo más significativo en una base de datos es el traslado de páginas desde el disco duro a la memoria RAM

El objetivo es responder las consultas con la menor cantidad posible de lecturas al disco duro

Se denomina **costo de I/O** al número de páginas llevadas a memoria para responder una consulta

Costo de una consulta

Disco Duro

Tupla 1	Tupla 2	Tupla 3	Tupla 4	Tupla 5	Tupla 6	Tupla 7	Tupla 8	Tupla 9
Página 1			**Página 2**			**Página 3**		

¿Cuál es el costo en I/O de la siguiente consulta?

```
SELECT * FROM T
```

$$\left\lceil \frac{|R|}{B} \right\rceil = \left\lceil \frac{9}{3} \right\rceil$$

El costo es 3, pues se leen las 3 páginas

Costo de una consulta

Disco Duro

Tupla 1	Tupla 2	Tupla 3	Tupla 4	Tupla 5	Tupla 6	Tupla 7	Tupla 8	Tupla 9
Página 1			**Página 2**			**Página 3**		

¿Cuál es el costo en I/O de la siguiente consulta?

```
SELECT T.a FROM T
```

$$\left\lceil \frac{|R|}{B} \right\rceil = \left\lceil \frac{9}{3} \right\rceil$$

El costo es nuevamente 3, pues se leen las 3 páginas

Costo de una consulta

Disco Duro

Tupla 1	Tupla 2	Tupla 3	Tupla 4	Tupla 5	Tupla 6	Tupla 7	Tupla 8	Tupla 9
Página 1			**Página 2**			**Página 3**		

¿Cuál es el costo en I/O de la siguiente consulta?

```
SELECT * FROM T WHERE T.a = 4
```

$$\left\lceil \frac{|R|}{B} \right\rceil = \left\lceil \frac{9}{3} \right\rceil$$

El costo es nuevamente 3, pues se leen las 3 páginas

Costo de una consulta

Disco Duro

Tupla 1	Tupla 2	Tupla 3	Tupla 4	Tupla 5	Tupla 6	Tupla 7	Tupla 8	Tupla 9
Página 1			**Página 2**			**Página 3**		

¿Cuál es el costo en I/O de las siguientes consultas?

```
SELECT * FROM T WHERE T.a = 4
SELECT * FROM T WHERE T.a >= 4
```

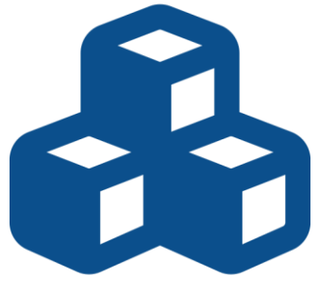
$$\left\lceil \frac{|R|}{B} \right\rceil$$



Costo de consulta

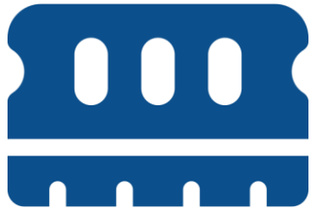
¿Es posible reducir este costo?

Mediante índices



Estructuras de datos

Antes: fundamentos de estructuras de datos



EEDD

Repaso: Memoria RAM

Memoria RAM

Un **bit** es la unidad indivisible de información computacional que puede valer 0 o 1. Además, un **bit** es una celda física indivisible de almacenamiento en un computador.

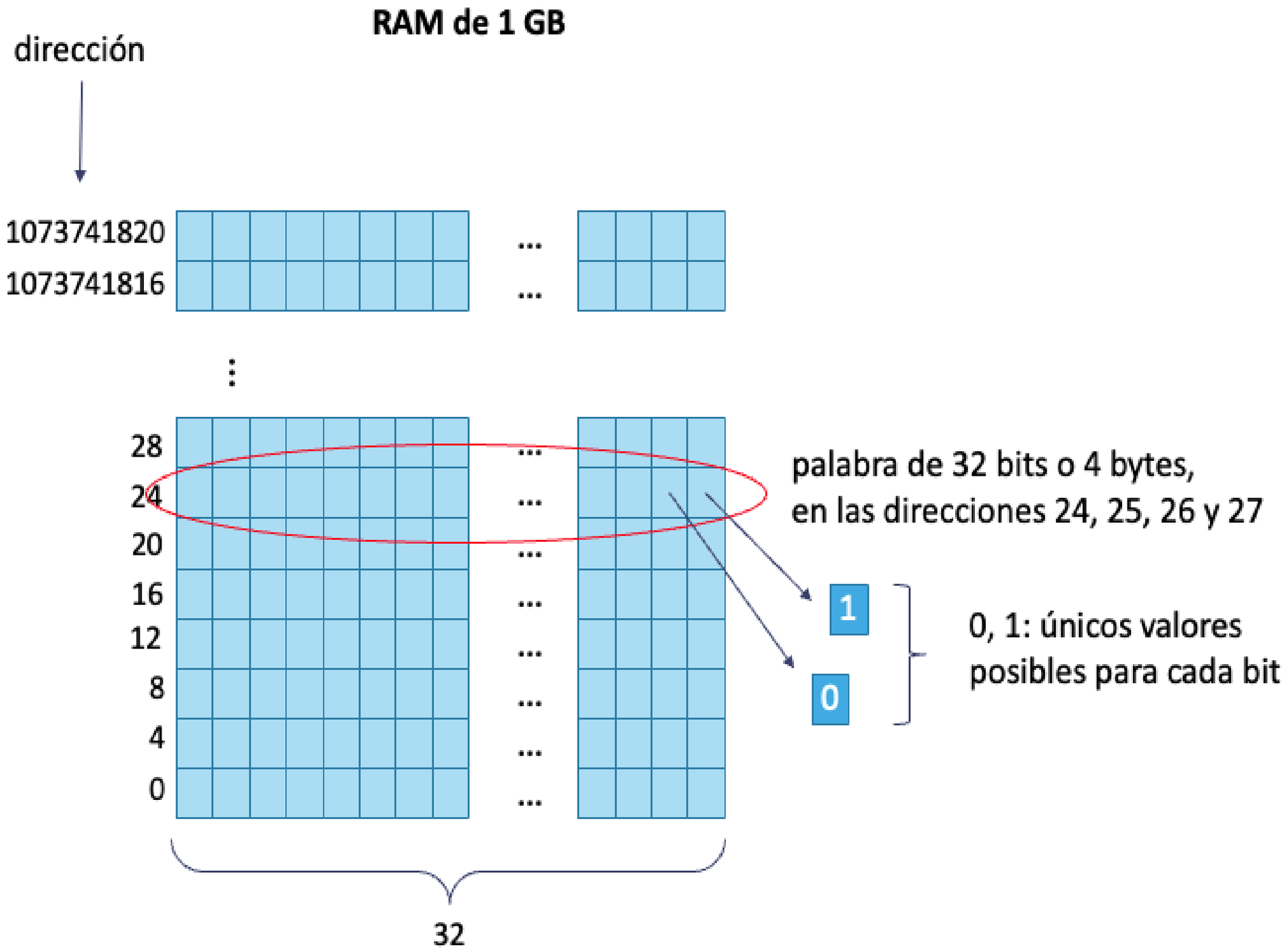
La memoria principal de un computador (RAM) puede representarse como una gran tabla, arreglo o matriz de bits:

- Cierto número de columnas
- Algunos miles de millones de filas

Cada fila de esta tabla tiene una dirección única:

- Corresponde a su posición relativa dentro de la tabla
- Es un natural que parte en 0 (dirección de la primera fila) y aumenta de 4 en 4

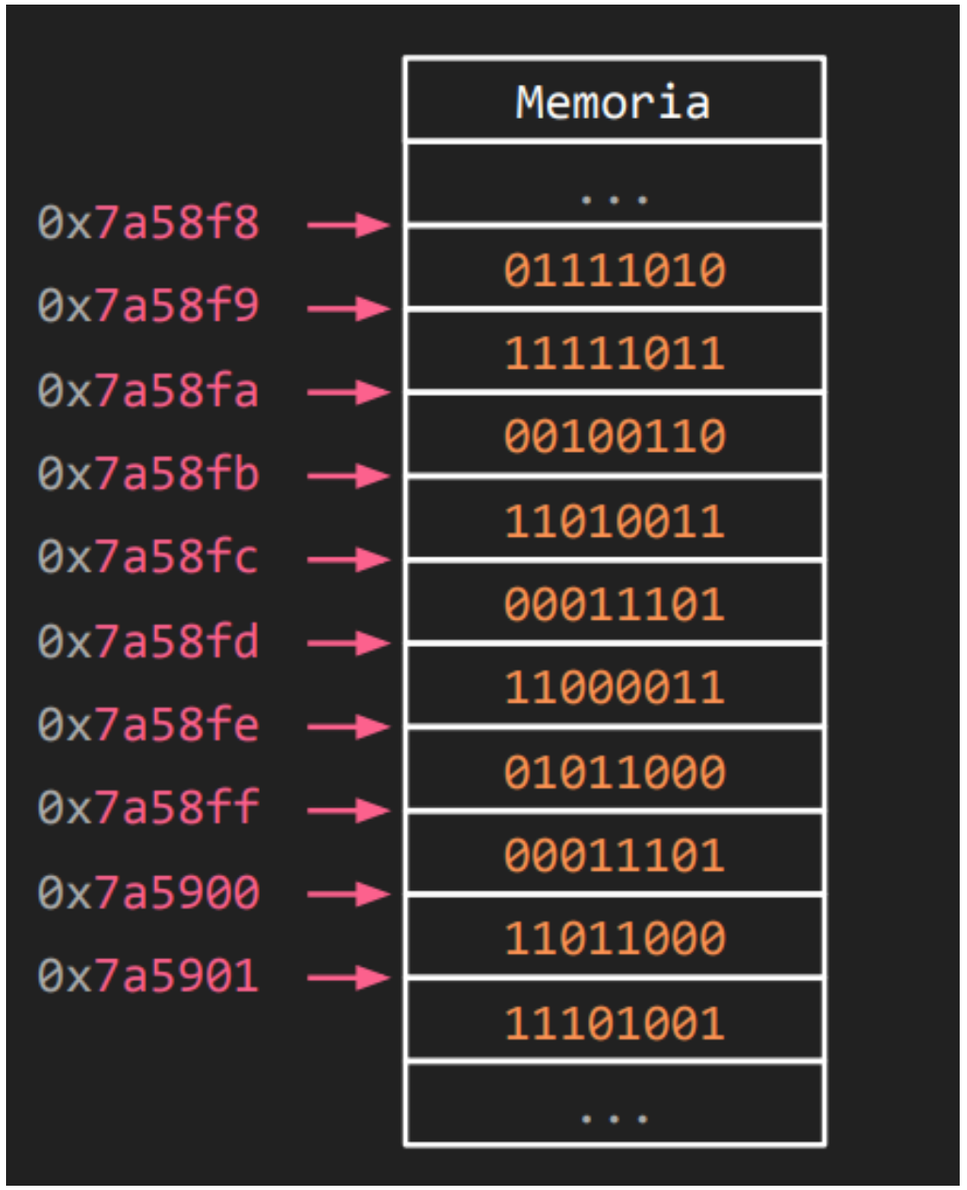
Memoria RAM


RAM de 1 GB
dirección
1073741820
1073741816
28
24
20
16
12
8
4
0
32
palabra de 32 bits o 4 bytes,
en las direcciones 24, 25, 26 y 27
1
0
0, 1: únicos valores
posibles para cada bit

Memoria RAM y Punteros

Cada variable de un programa tiene:

- Posición (**dirección de memoria**)
- Tamaño, en número de bytes
- **Valor**



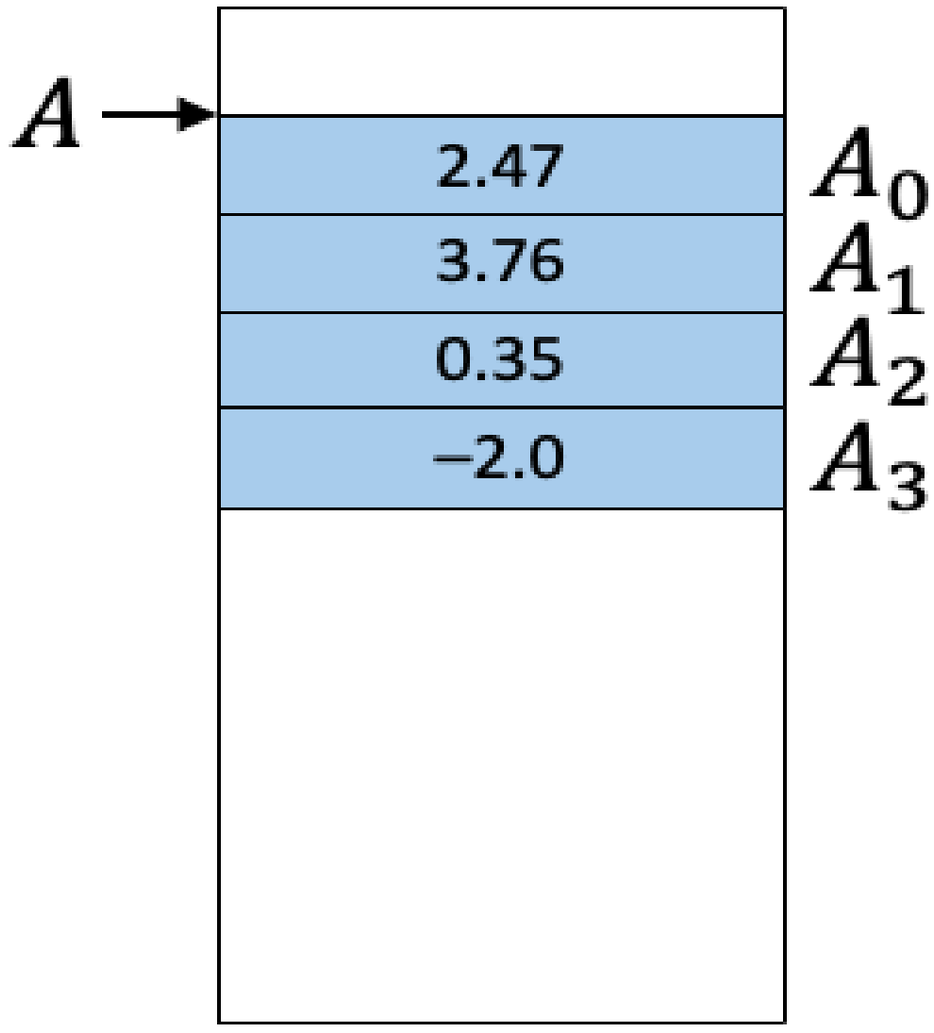
Puede pensarse en la memoria como una tabla gigante que asocia direcciones a bytes

Un **puntero** es una variable cuyo **valor** es la **dirección** de memoria de otra variable.

Arreglos

Un **arreglo** es una secuencia de celdas que tiene largo fijo:

- Cada celda es del mismo tamaño
- Almacenan valores del mismo tipo



Se almacena en memoria de manera contigua

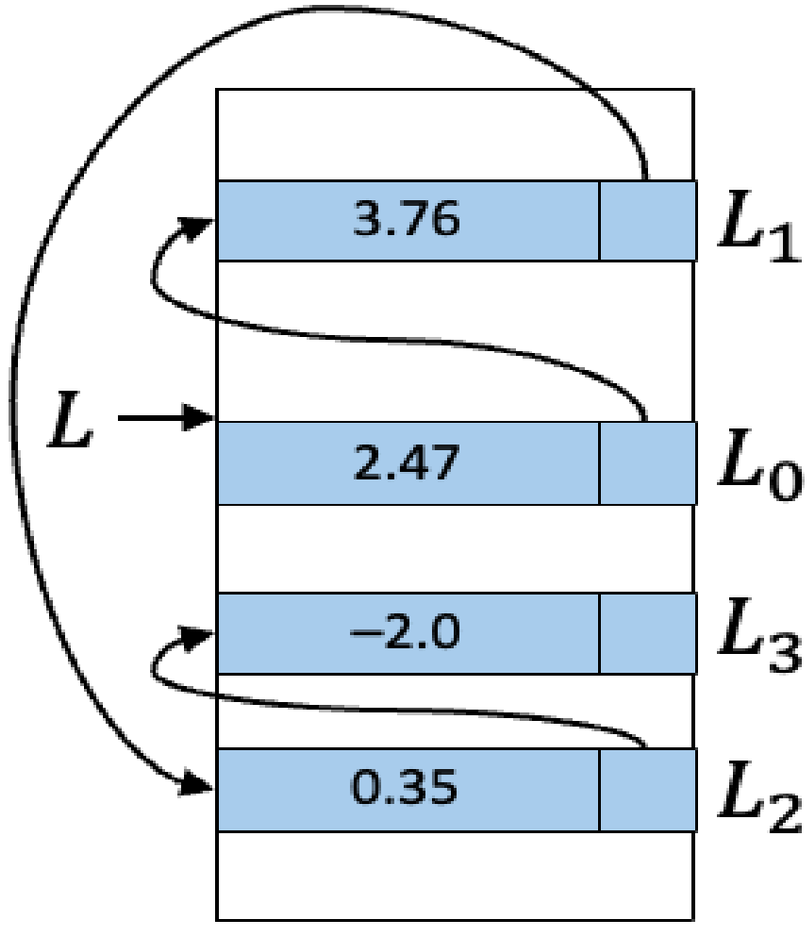
	2.47	3.76	0.35	−2.0	

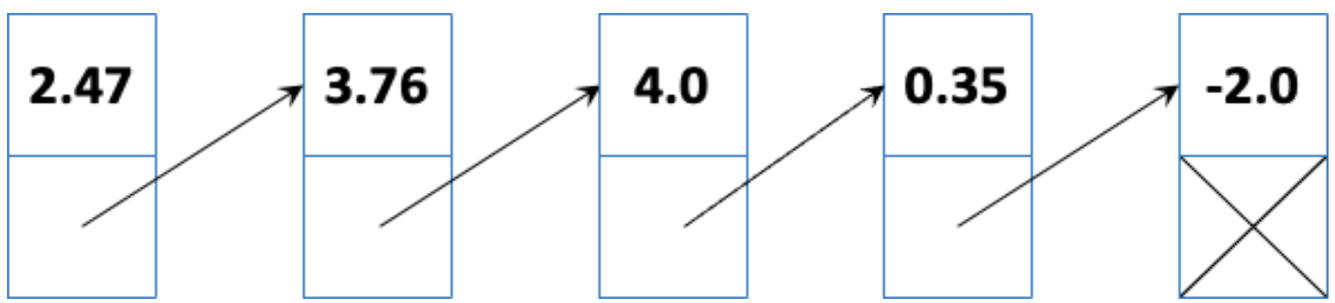
Listas Ligadas

Una lista ligada es una secuencia de celdas que tiene largo variable

- Cada celda es del mismo tamaño

Se almacena en memoria de manera aleatoria, utilizando **punteros**







EEDD

Tablas de Hash

Funciones de Hash

Dado un espacio de llaves K y un natural m > 0, una función de hash se define como

$h : K \rightarrow \{0, \ldots, m - 1\}$

Dado $k \in K$, se denomina valor de hash de k a la evaluación h(k). Obsérvese que:

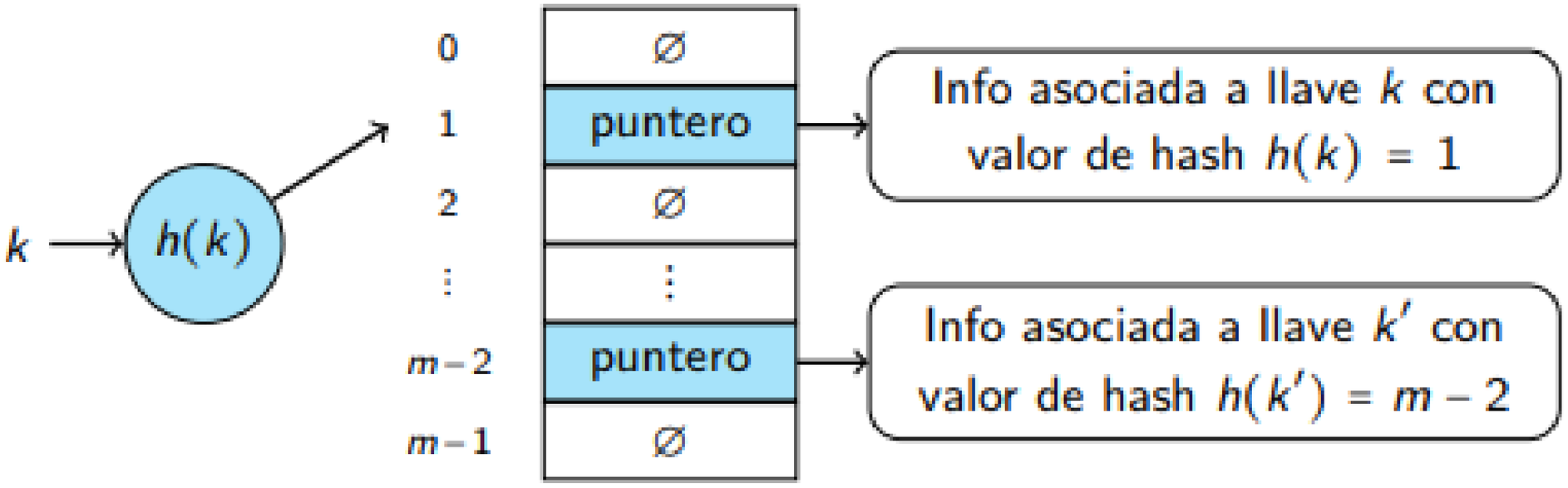
- Una función de hash permite mapear un espacio de llaves a otro más pequeño (con m razonable)
- Una función de hash no necesariamente es inyectiva
- Si m < | K | , no puede ser inyectiva.
- En la práctica, m << | K |

Tablas de Hash

Dado m > 0 y un conjunto de llaves K , una tabla de hash A es una EDD que asocia valores a llaves indexadas usando una función de hash

$h : K \rightarrow \{0, \ldots, m - 1\}$

Se dice que A es de tamaño m.



Tablas de Hash

Hashing general:

```
HashSearch  (A, k):
  return A[h(k)]

HashInsert  (A, k, v ):
  A[h(k)] = v

HashDelete  (A, k):
  A[h(k)] = ∅
```

Sin embargo, h no necesariamente es inyectiva

Es decir, puede ocurrir una colisión h(k1) = h(k2) para k1 ≠ k2

Tablas de Hash

Para ejemplificar el problema de las colisiones, considérese la siguiente función de hash:

$$h(k) = k \bmod m$$

Se conoce como hashing modular y corresponde al resto al dividir k entre m. Obsérvese que $h(k) \in \{0, \ldots, m-1\}$ para todo k. Todas las llaves con el mismo resto al dividir entre m generan una colisión, i.e.

$h(k1) = h(k2) \Leftrightarrow k1 \equiv m\ k2$

Ejemplo. Tomando m = 100 y K = {0, . . . , 999}, el hashing modular cumple:

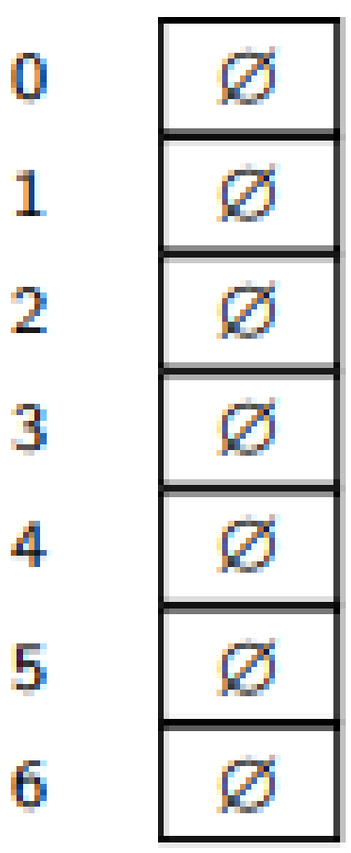
$h(12) = h(112) = \ldots = h(912) = 12$

$h(18) = \ldots = h(918) = 18$

Tablas de Hash – Inserciones

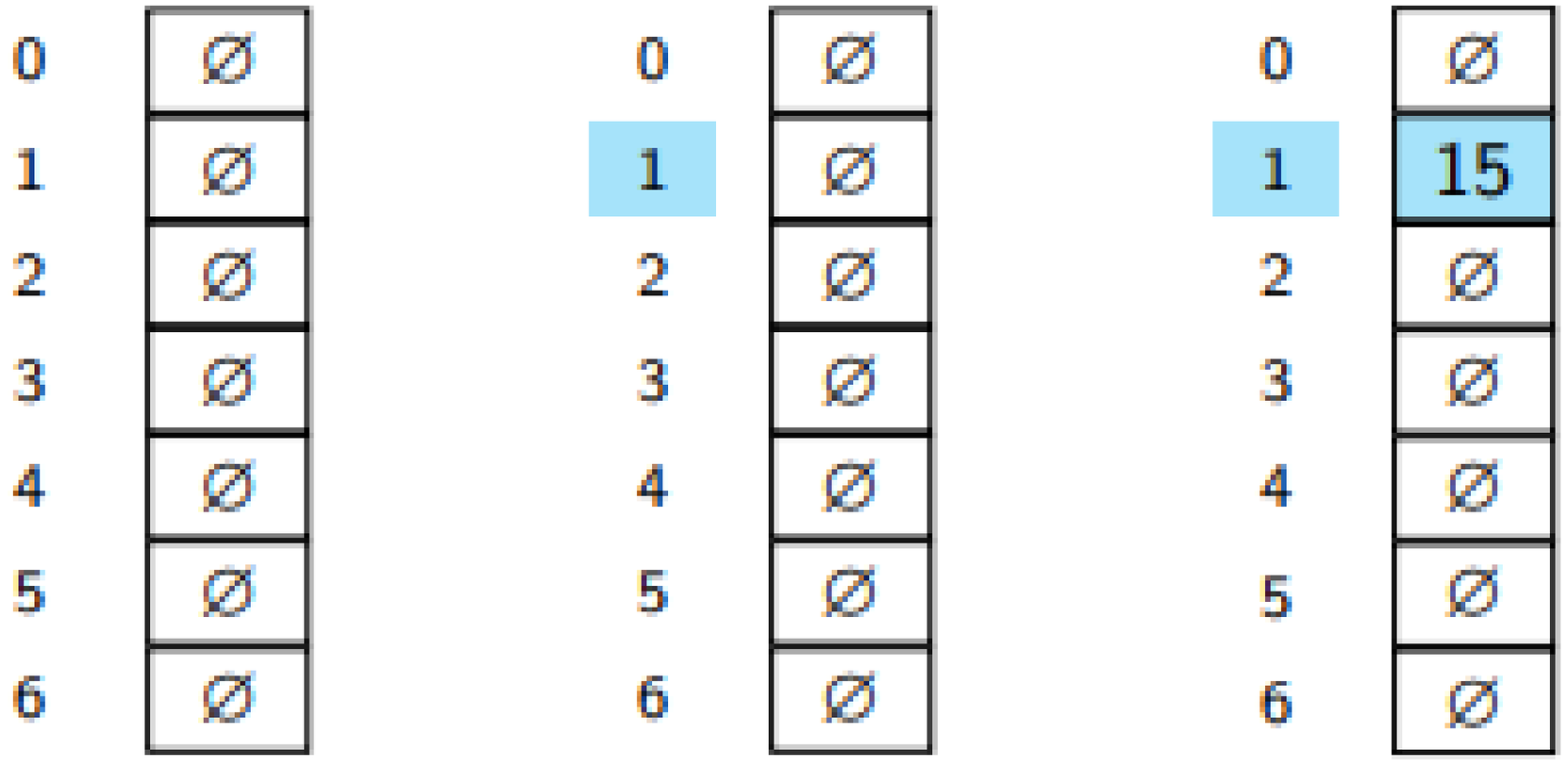
Se utilizará la función de hashing modular para ilustrar las inserciones:

Considérese m = 7. Se inserta la llave 15 en la siguiente tabla de hash. Su valor de hash es h(15) = 15 mod 7 = 1



Tablas de Hash – Inserciones

La posición $h(15) = 1$ está libre, por lo que se almacena la llave



Tablas de Hash – Inserciones

A continuación se inserta la llave 37:

Su valor de hash es h(37) = 37 mod 7 = 2

0	∅
1	15
2	∅
3	∅
4	∅
5	∅
6	∅

0	∅
1	15
2	∅
3	∅
4	∅
5	∅
6	∅

0	∅
1	15
2	37
3	∅
4	∅
5	∅
6	∅

Tablas de Hash – Inserciones

A continuación se inserta la llave 51

Su valor de hash es h(51) = 51 mod 7 = 2

0	∅
1	15
2	37
3	∅
4	∅
5	∅
6	∅

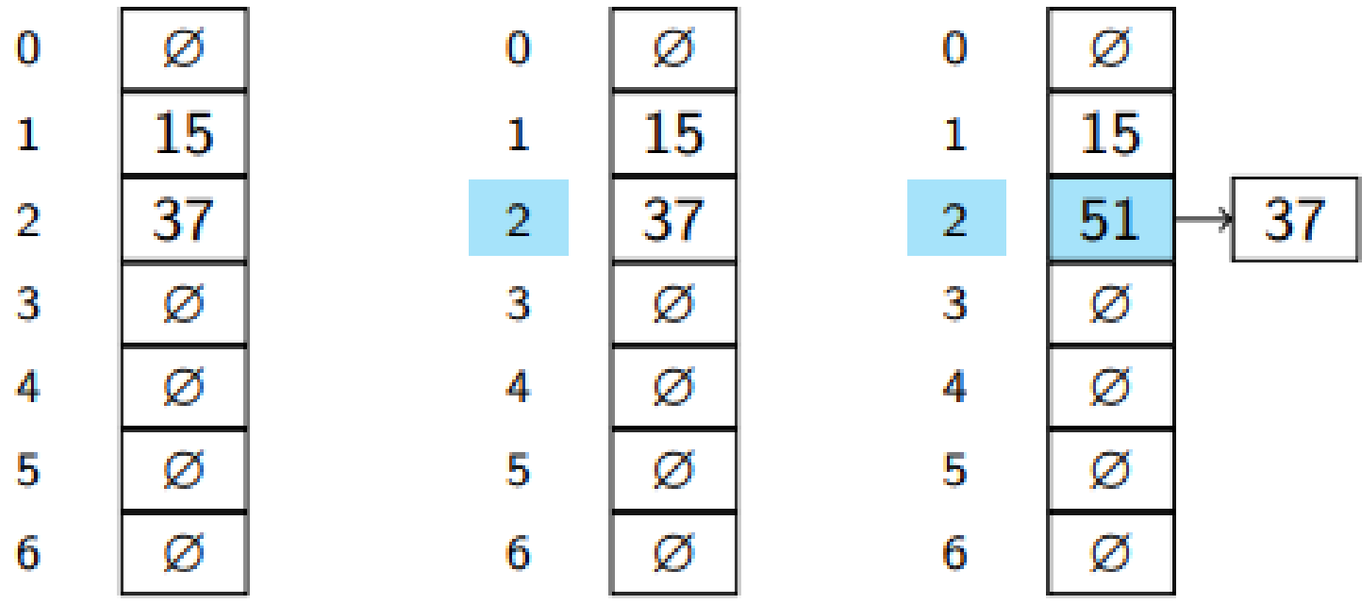
0	∅
1	15
2	37
3	∅
4	∅
5	∅
6	∅

¿Cómo se resuelve la colisión?

Tablas de Hash – Inserciones

Primera propuesta: encadenamiento.

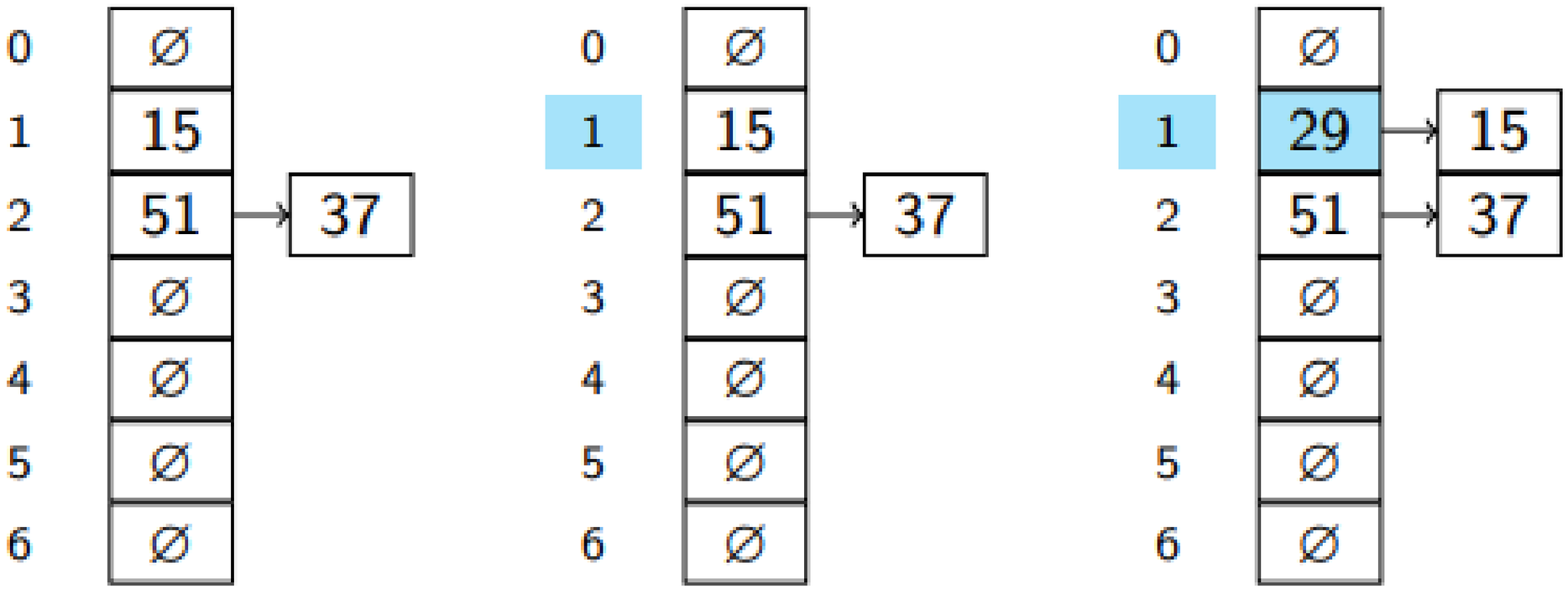
- Cada valor guardado es un nodo de una lista ligada
- Cada colisión agrega un nodo al principio/final de la lista



Tablas de Hash – Inserciones

Al insertar la llave 29 se aplica el mismo procedimiento

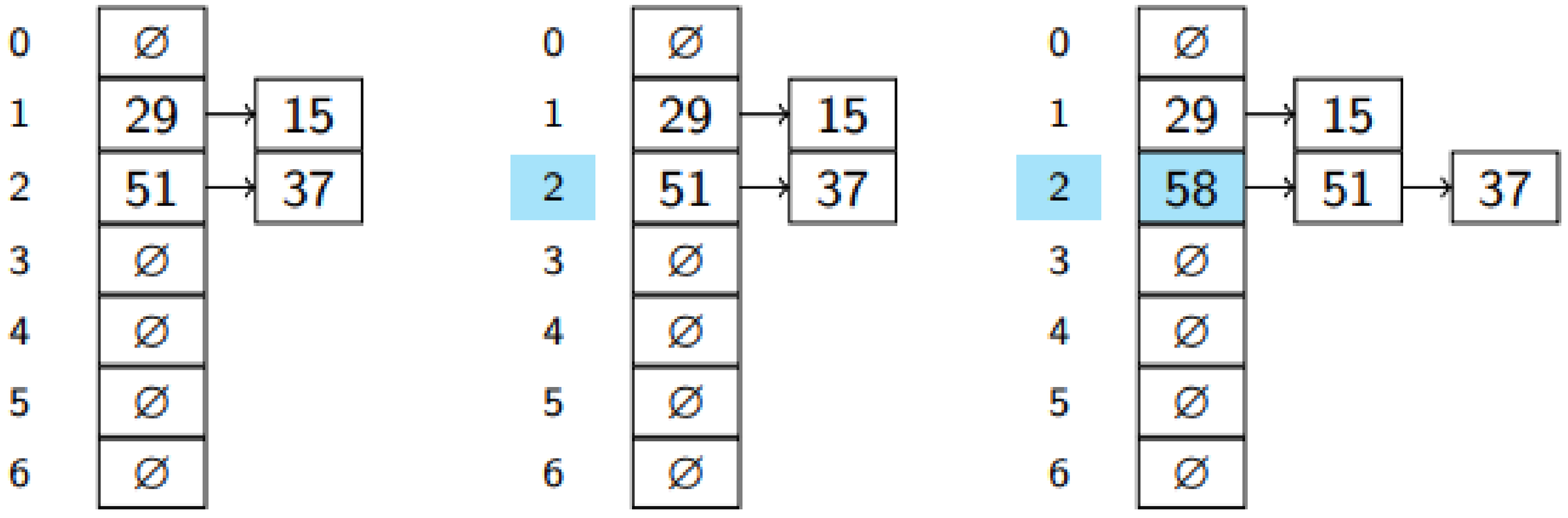
Su valor de hash es h(29) = 29 mod 7 = 1



Tablas de Hash – Inserciones

Al insertar la llave 58 se aplica el mismo procedimiento

Su valor de hash es h(58) = 58 mod 7 = 2



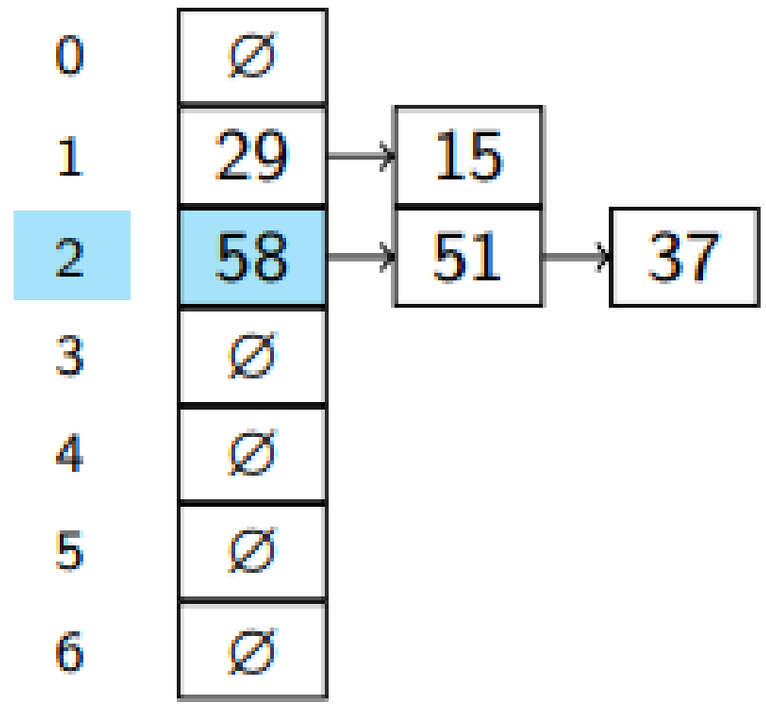
Tablas de Hash – Inserciones

Las operaciones de diccionario involucran la lista ligada A[h(k)]

`ChainedHashSearch (A, k):`
Buscar llave k en A[h(k)]

`ChainedHashInsert (A, k, v ):`
Insertar (k, v) como cabeza de A[h(k)]

`ChainedHashDelete (A, k):`
Eliminar llave k de A[h(k)]



La complejidad de estas operaciones depende de qué tan largas sean las listas.
Una buena función de hash distribuye las llaves de forma homogénea

Tablas de Hash – Factor de carga

Dado que las colisiones impactan la tabla, interesa medir cuántos datos hay almacenados.

Dada una tabla de hash A de tamaño m con n valores almacenados, se define su **factor de carga** como

$$\lambda = n/m$$

El factor de carga es una medida de qué tan llena está la tabla.

Según la estrategia de resolución de colisiones:

Encadenamiento: es aceptable $\lambda \approx 1$

Direccionamiento abierto: $\lambda > 0.5$ resulta en inserciones y búsquedas muy lentas



EEDD

B+ Tree

B+ Tree



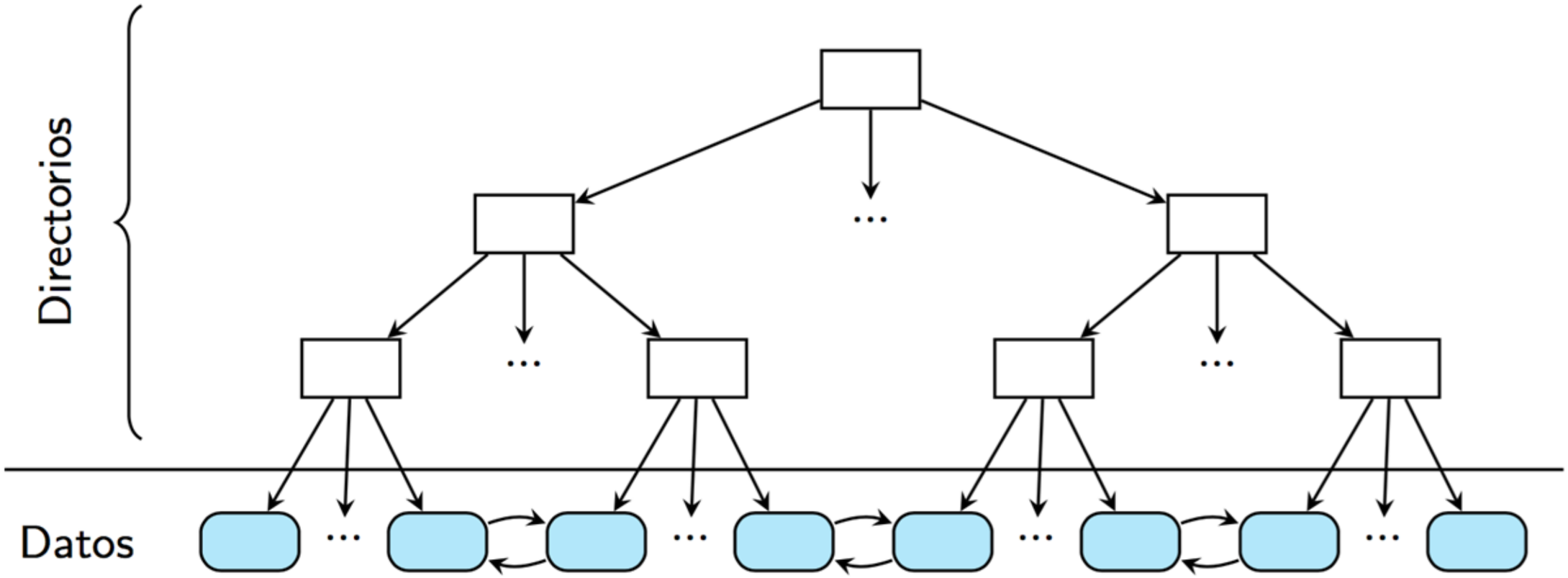
“B+ Trees are by far the most important access path structure in database and file systems”, Gray y Reuter (1993).

B+ Tree

Es un índice basado en un árbol:

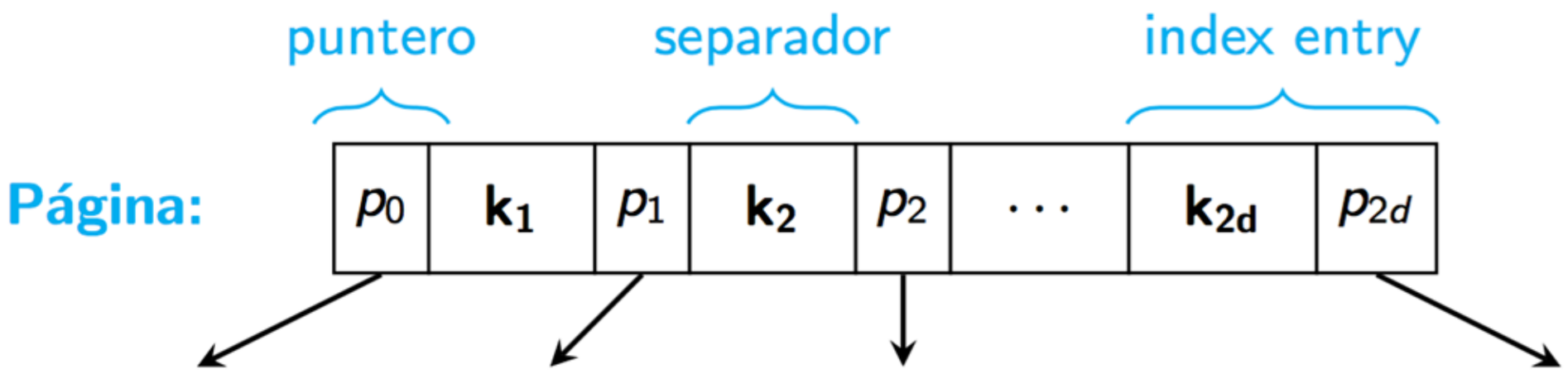
- Las páginas del disco representan nodos del árbol
- **Las tuplas** están en las hojas
- Provee un tiempo de búsqueda logarítmico
- Se comporta bien en consultas de rango
- Es el índice que usa Postgres sobre las llaves primarias (y en general, todos los sistemas)

B+ Tree



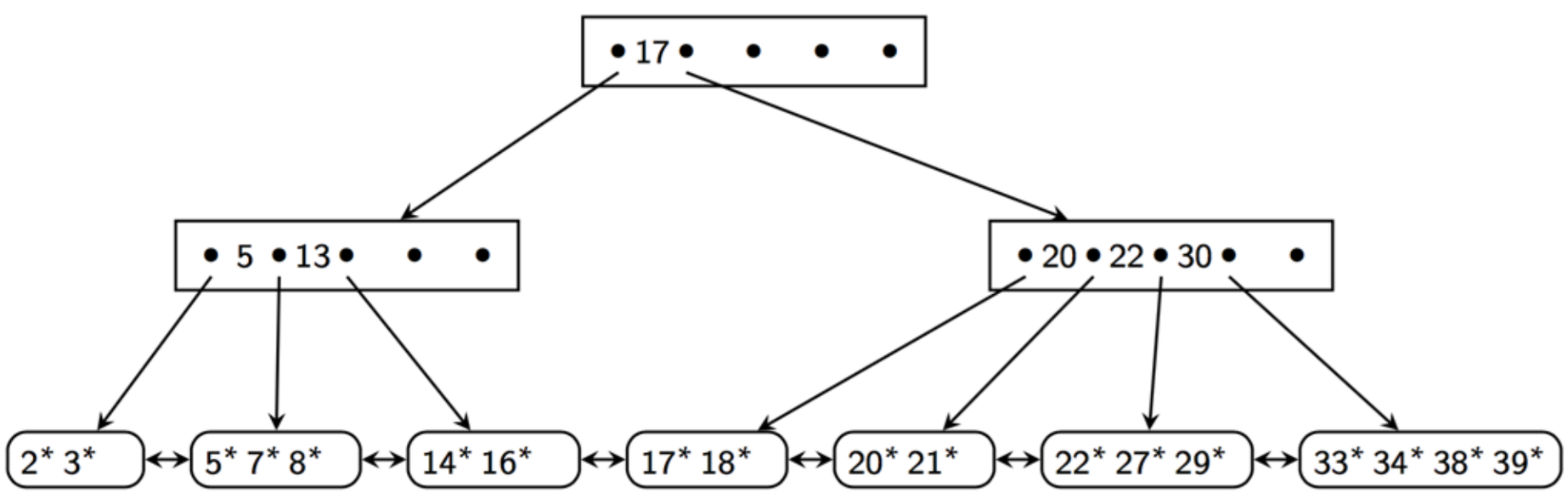
B+ Tree

Cada nodo del directorio tiene la siguiente forma:



B+ Tree Index

En los directorios, los nodos tienen una **Search Key** y un puntero antes y después de ella

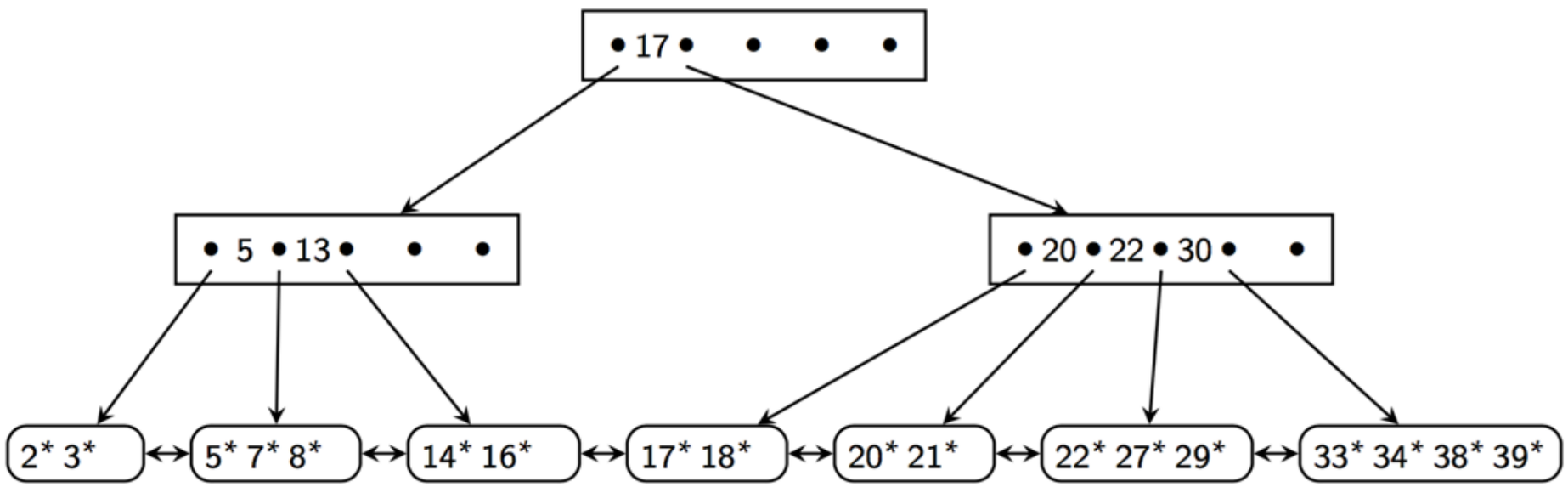


B+ Tree

Ejemplo: consulta por un valor

Supóngase que este índice también almacena artistas indexados por su aid

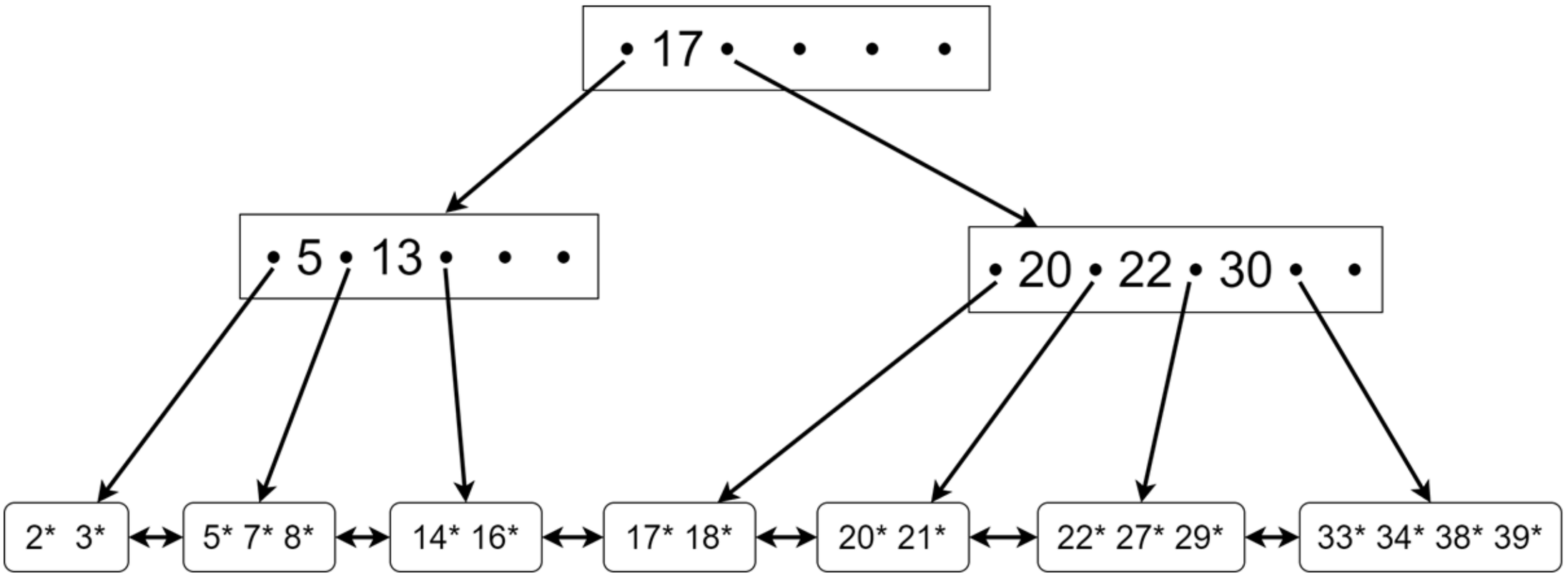
Se busca el artista con aid = 21



B+ Tree

Ejemplo: consulta por un valor

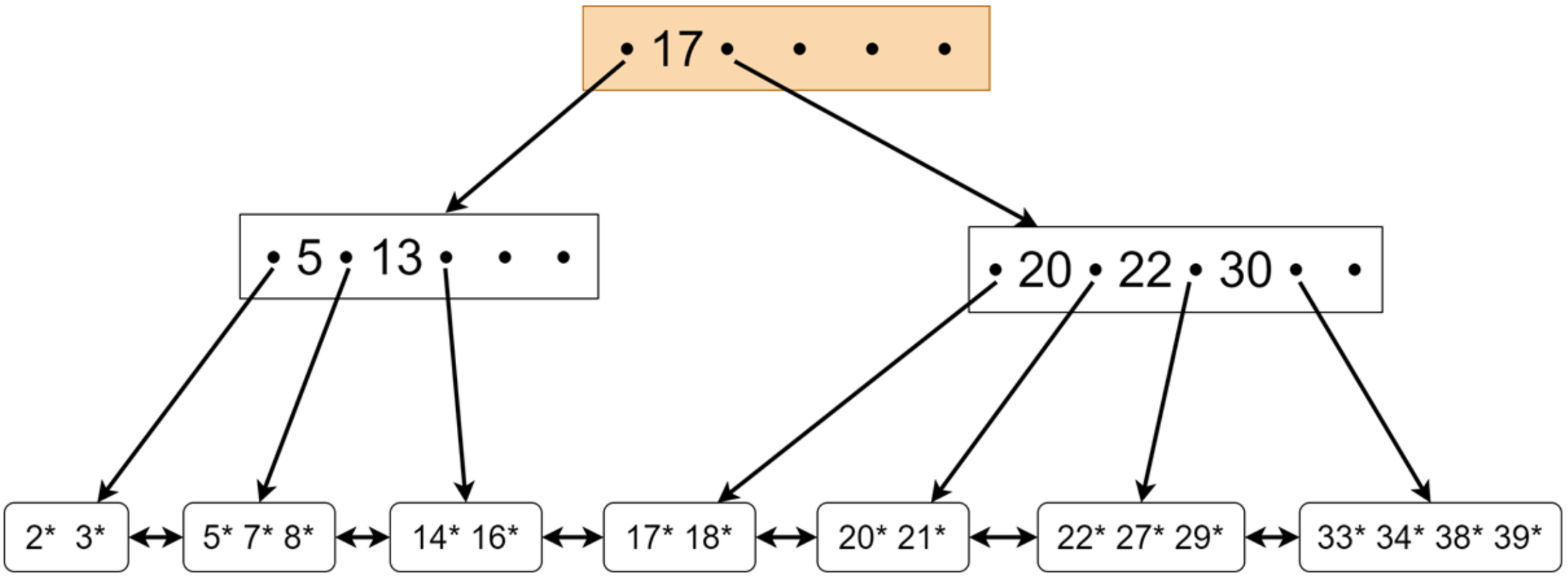
Se busca el artista con aid = 21



B+ Tree

Ejemplo: consulta por un valor

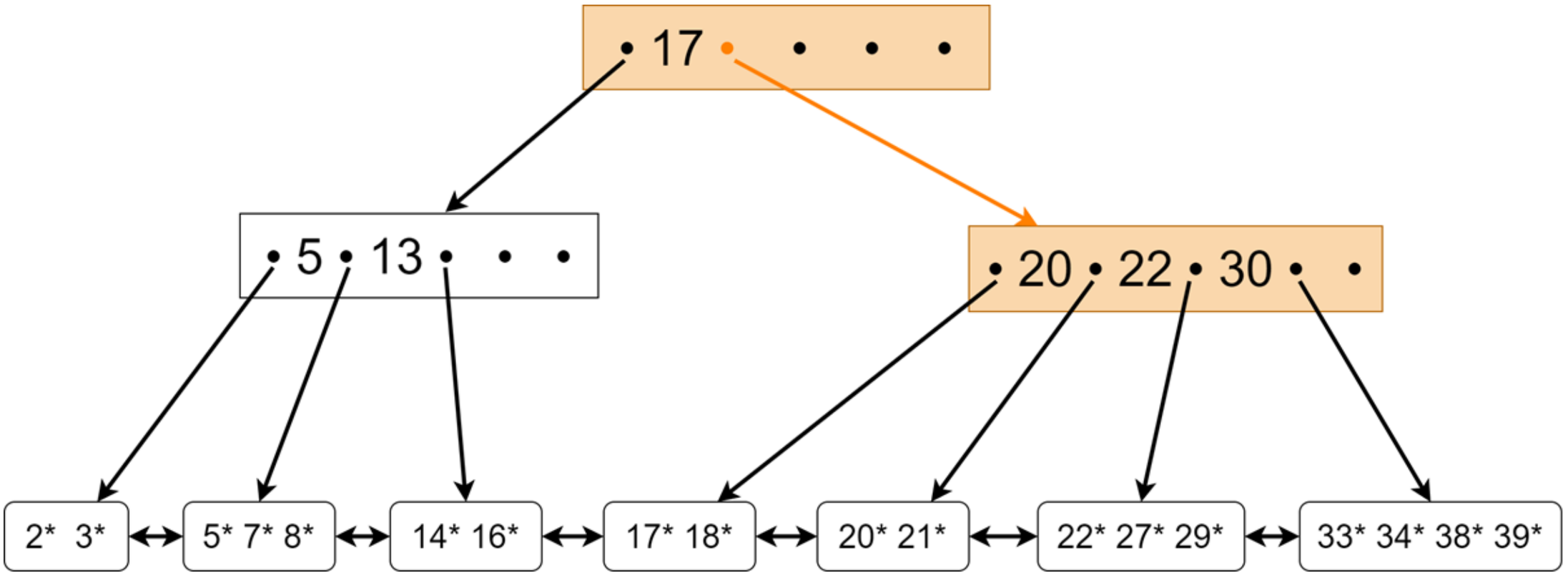
Se busca el artista con aid = 21



B+ Tree

Ejemplo: consulta por un valor

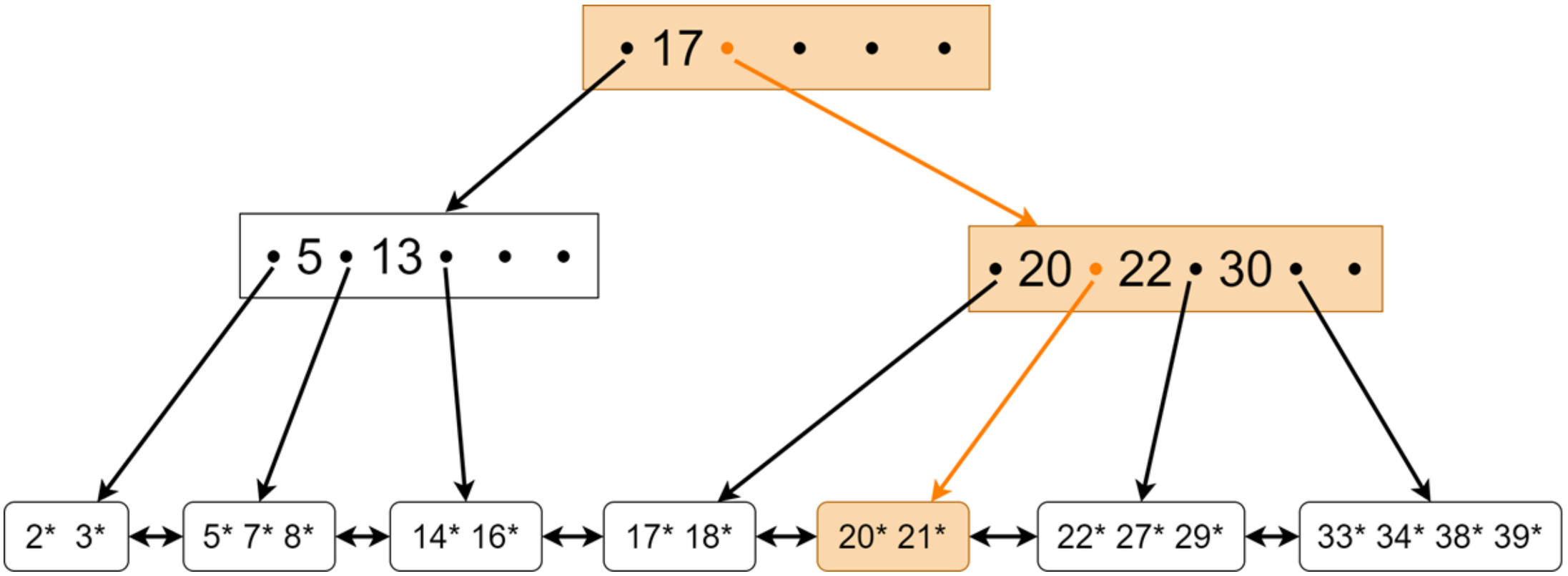
Se busca el artista con aid = 21



B+ Tree Index

Ejemplo: consulta por un valor

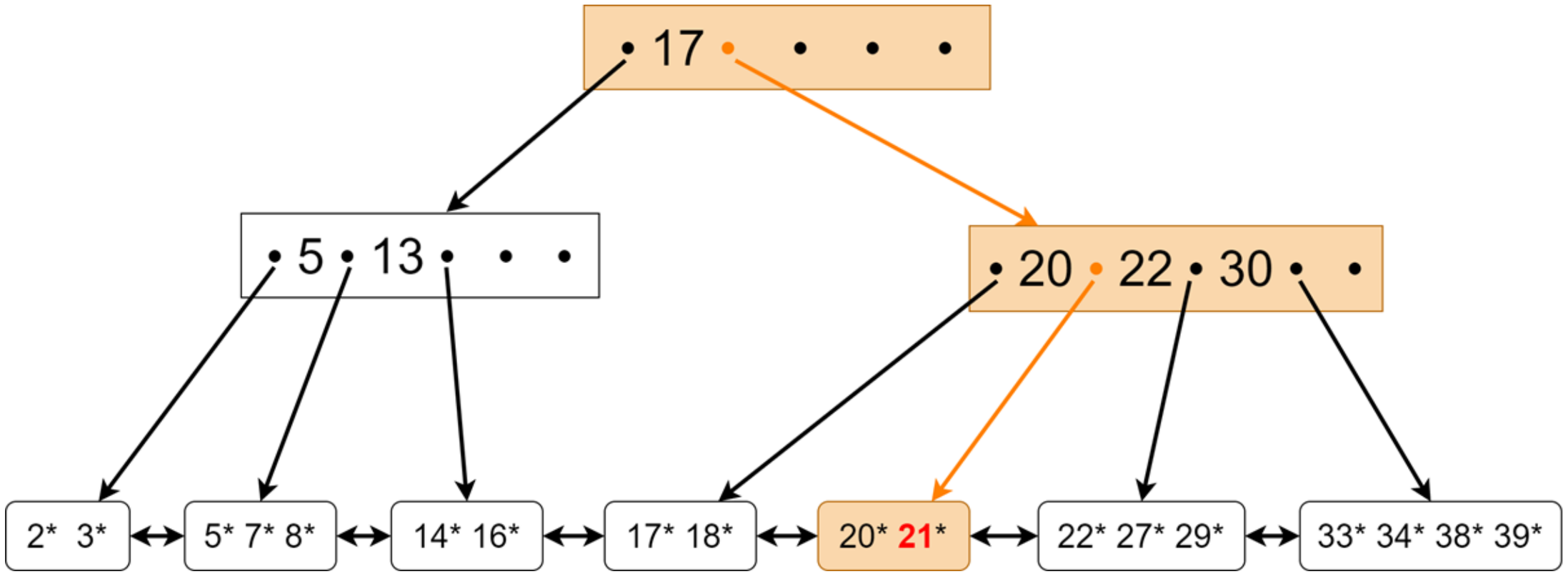
Se busca el artista con aid = 21



B+ Tree

Ejemplo: consulta por un valor

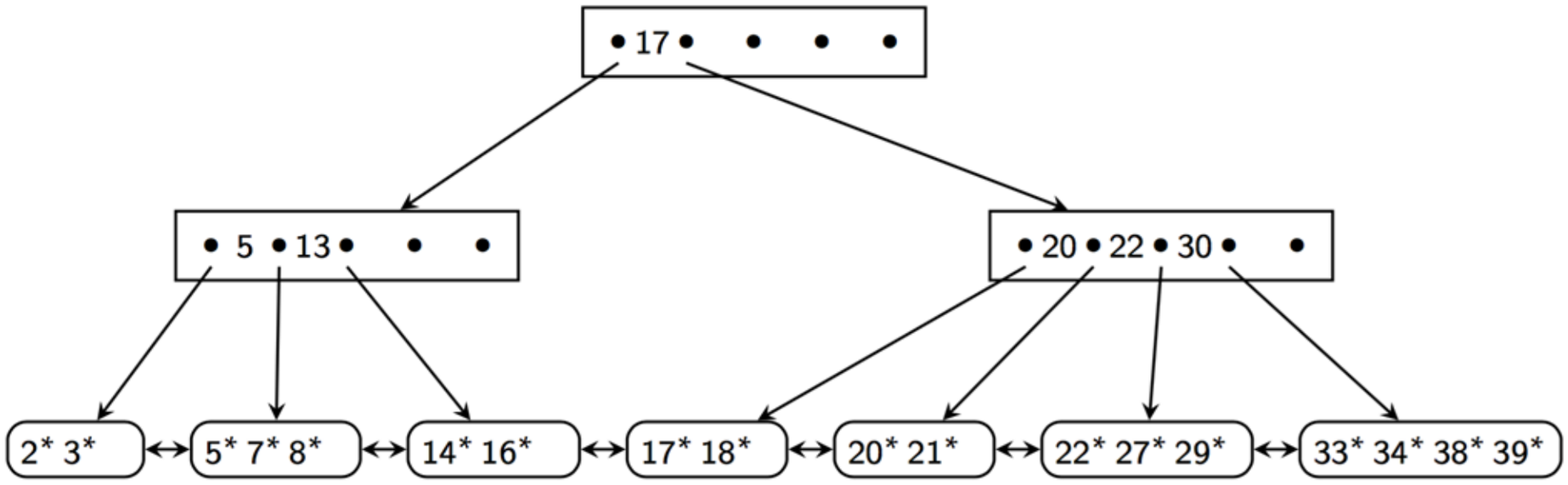
Se busca el artista con aid = 21



¿Es útil esta estructura en consultas de rango?

B+ Tree

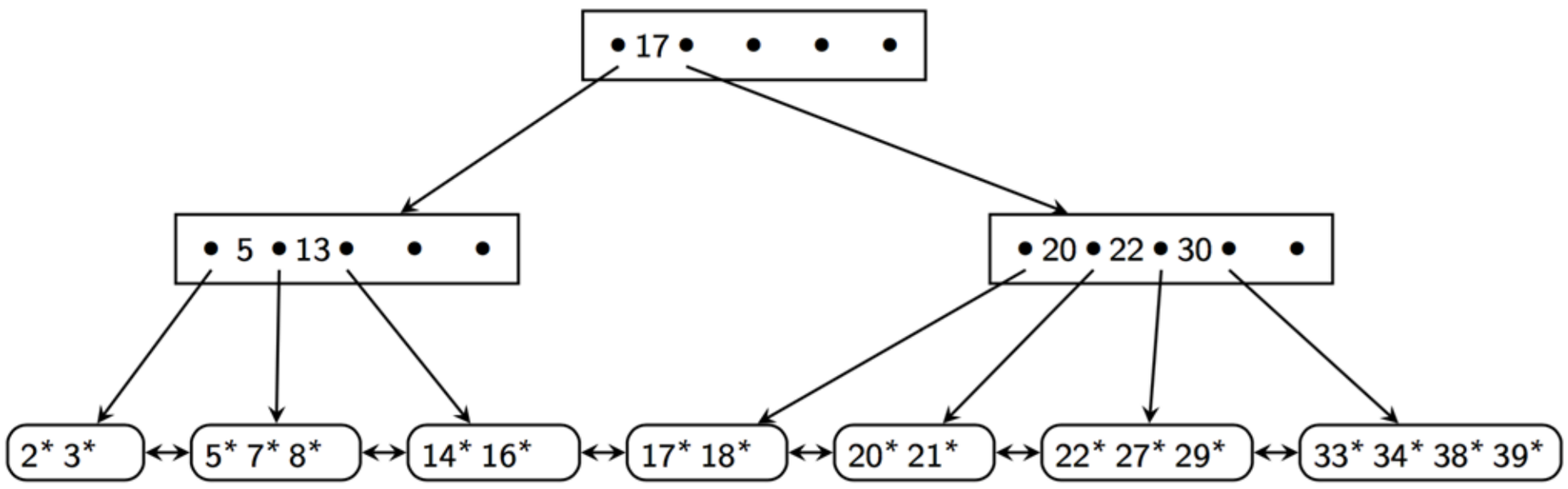
Las tuplas están ordenadas según el atributo indexado y, además, las hojas están encadenadas entre sí



B+ Tree

Ejemplo: consulta de rango

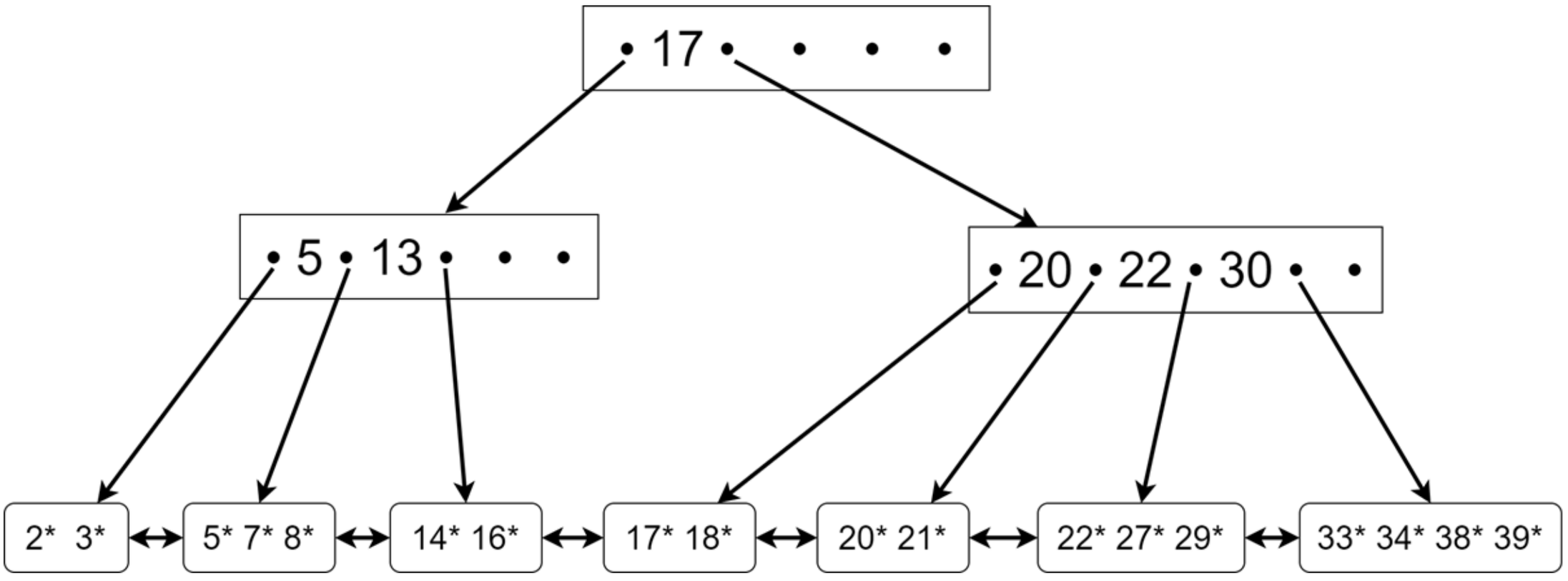
Considérese ahora la consulta de todos los artistas con aid entre 5 y 26



B+ Tree

Ejemplo: consulta de rango

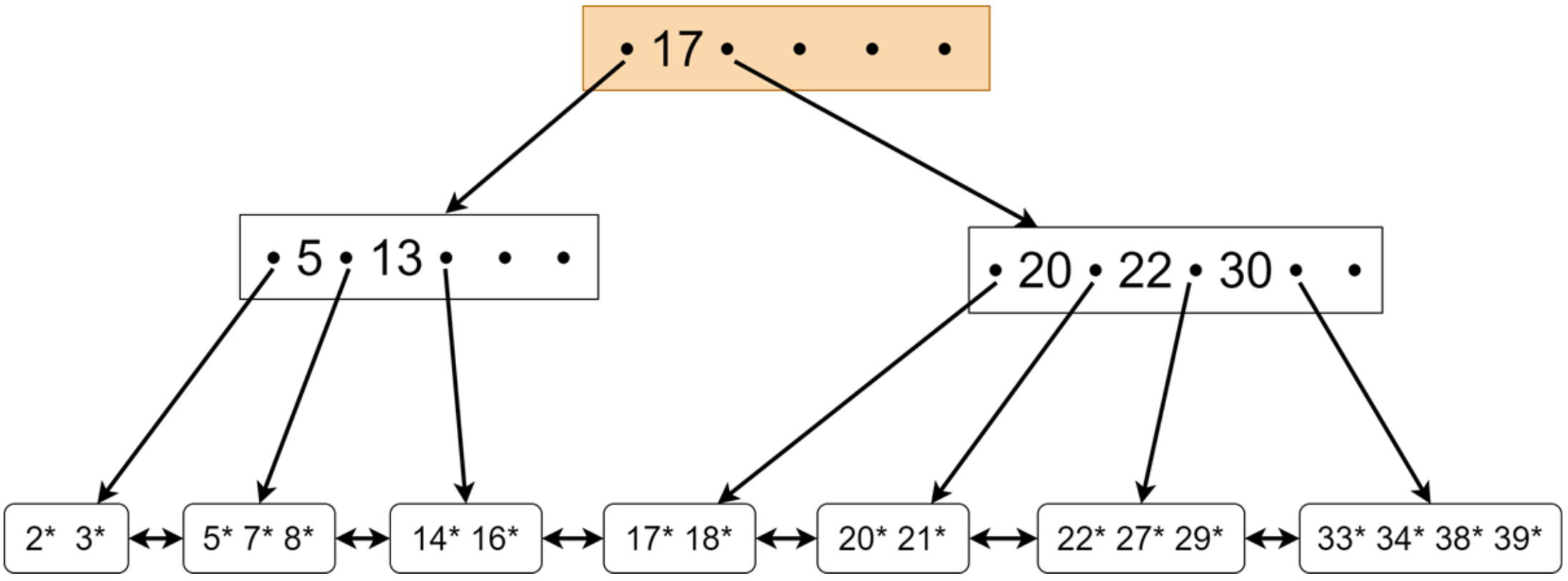
Todos los artistas con aid entre 5 y 26



B+ Tree

Ejemplo: consulta de rango

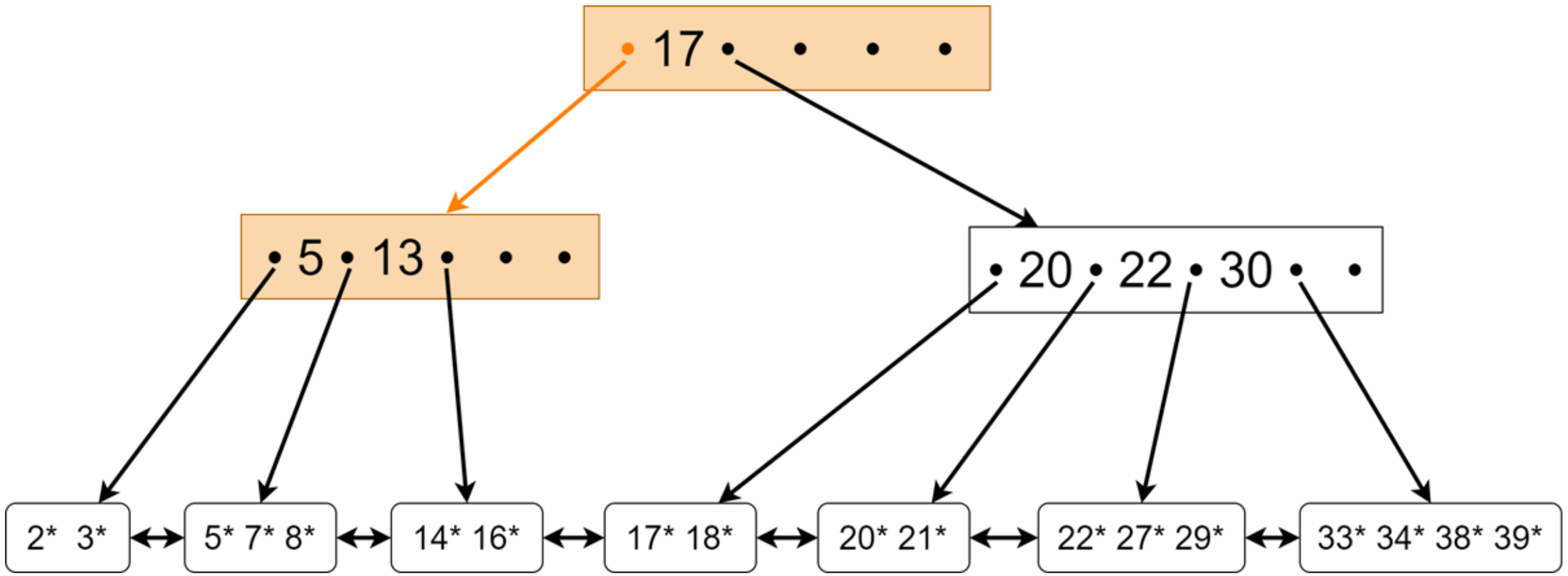
Todos los artistas con aid entre 5 y 26



B+ Tree

Ejemplo: consulta de rango

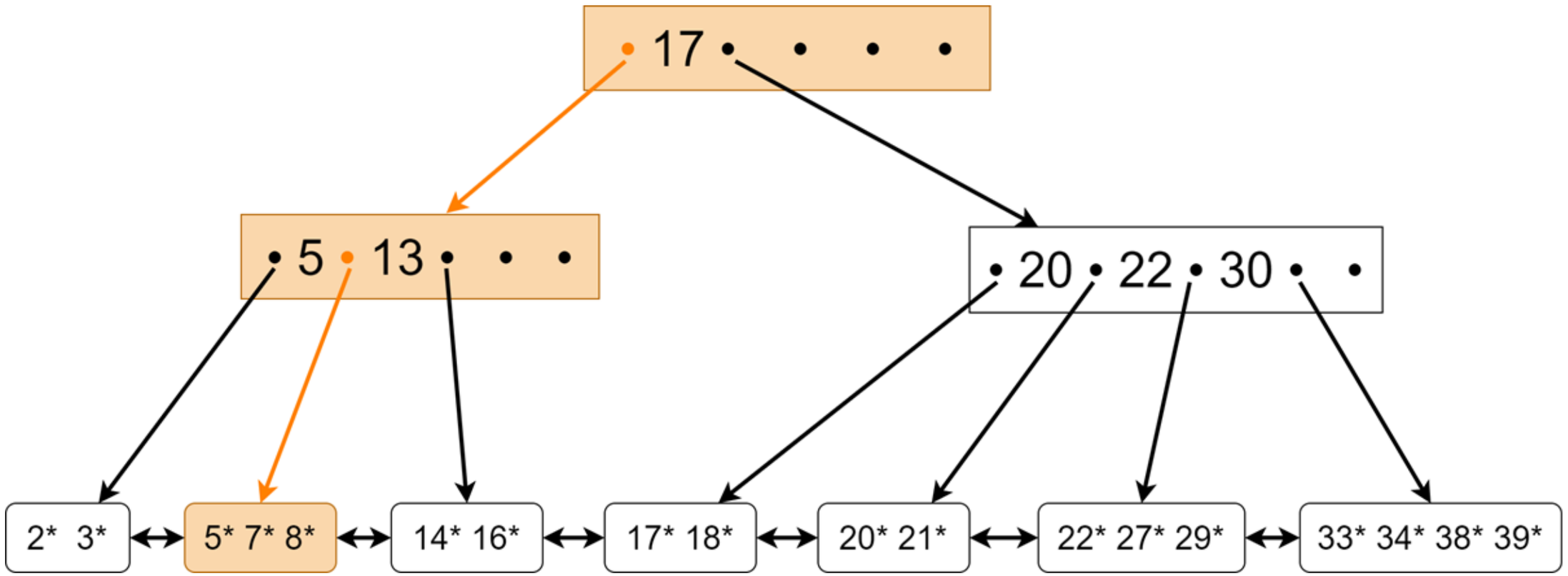
Todos los artistas con aid entre 5 y 26



B+ Tree

Ejemplo: consulta de rango

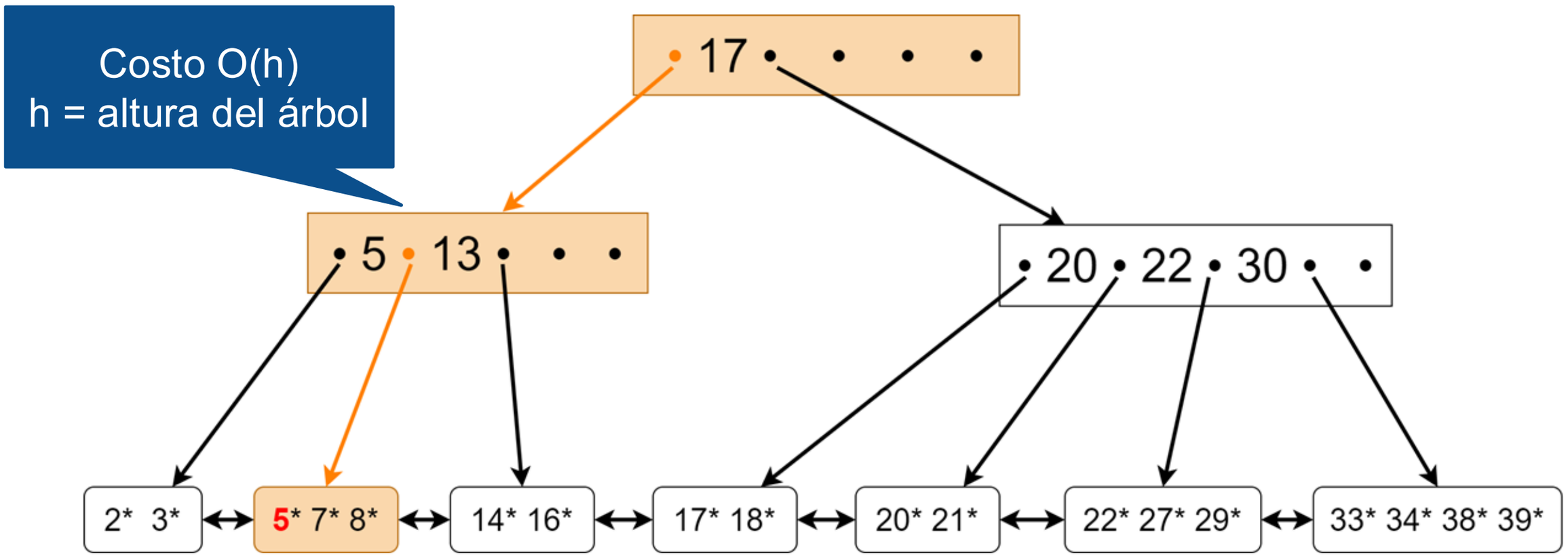
Todos los artistas con aid entre 5 y 26



B+ Tree

Ejemplo: consulta de rango

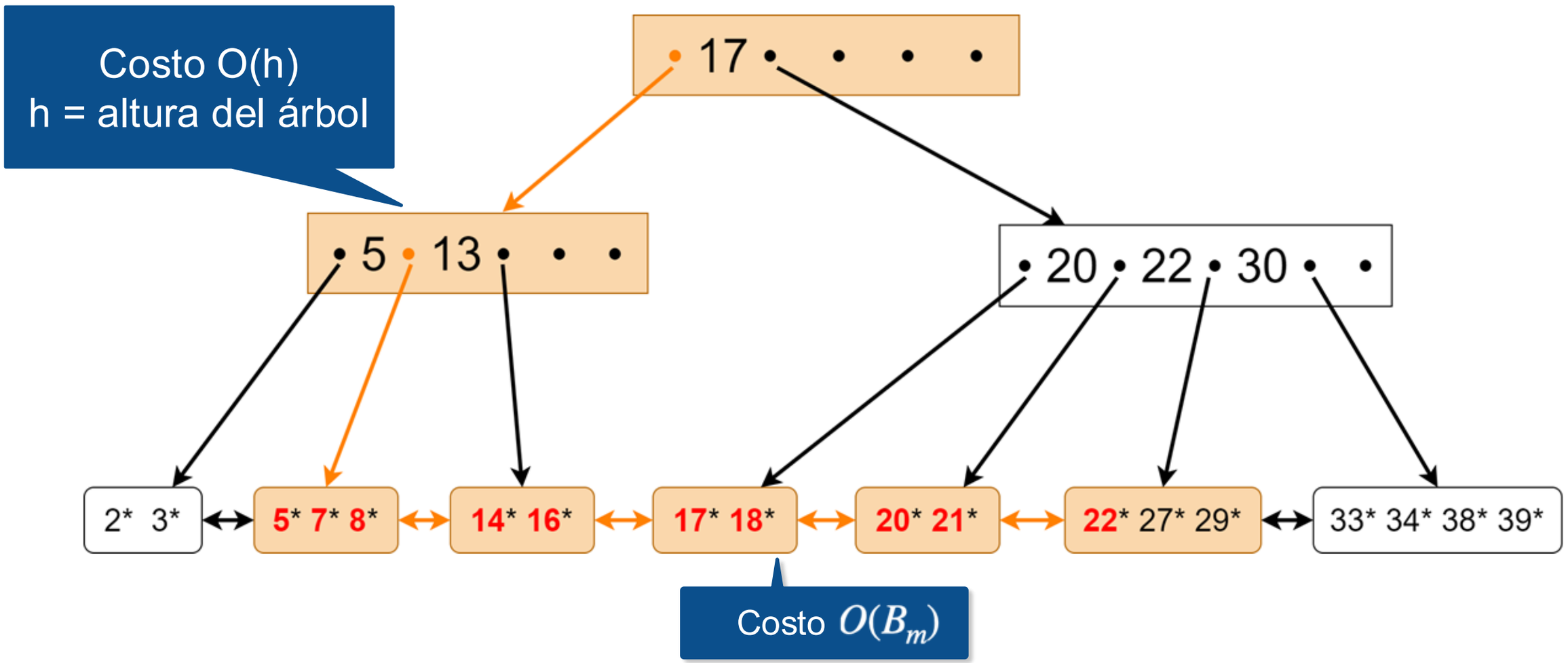
Todos los artistas con aid entre 5 y 26



B+ Tree

Ejemplo: consulta de rango

Todos los artistas con aid entre 5 y 26



B_m : nr de bloques con récords que calzan búsqueda

Índices

Estructuras que optimizan el acceso a los datos para una consulta o conjunto de consultas en particular

A un índice se le solicitan las tuplas con cierto valor en un atributo (o con un valor en cierto rango)

El índice encontrará rápidamente las tuplas que cumplan con la condición

(Idealmente un índice debe caber en RAM)

Consultas a Optimizar

Consultas **por valor**

```
SELECT *
FROM Table
WHERE Table.value = ‘value’
```

Consultas **por rango**

```
SELECT *
FROM Table
WHERE Table.value >= ‘value’
```

Índices

```
CREATE INDEX <nombre índice> ON <table> <atributo>
```

Las llaves primarias están indexadas por defecto

Considérese la tabla:

```
Usuarios( uid: int PRIMARY KEY,
          nombre: text,
          edad: int)
```

donde la tabla está indexada por la primary key, que es el atributo uid

Índices

El índice normalmente es una colección de archivos que puede ir completo en memoria

Si se ejecuta la consulta:

```
SELECT * FROM Usuarios WHERE uid = 104
```

esta se resolverá mediante el índice, lo que evita recorrer toda la tabla

Índices

Definiciones

Search Key: es el parámetro con el que se realiza una búsqueda en el índice (por ejemplo, el uid de una tabla de usuarios)

Data Entry: la tupla que retorna el índice al consultar por una Search Key en particular

Por ahora:
Data Entry = la tupla

Ejemplo: EDD aplicadas a BDD

Tablas de Hash

Se observan colisiones, esto es, dos valores que van a parar al mismo casillero

En general, se supone que las funciones de hash distribuyen de manera uniforme

También se supone que hay suficientes casilleros para que la búsqueda se haga en 1 paso (o no mucho más)

Método: Hash Index

La idea es replicar el concepto de las tablas de hash pero en un sistema de bases de datos

Aquí, los casilleros corresponden a páginas del disco duro

En cada página caben muchas tuplas

Método: Hash Index

Recuérdese que el objetivo es encontrar rápidamente tuplas para consultas de igualdad

```
SELECT * FROM Artistas A WHERE A.aid = 12
```

En este caso, rápido significa hacer la menor cantidad de I/O posible

Método: Hash Index

Ejemplo

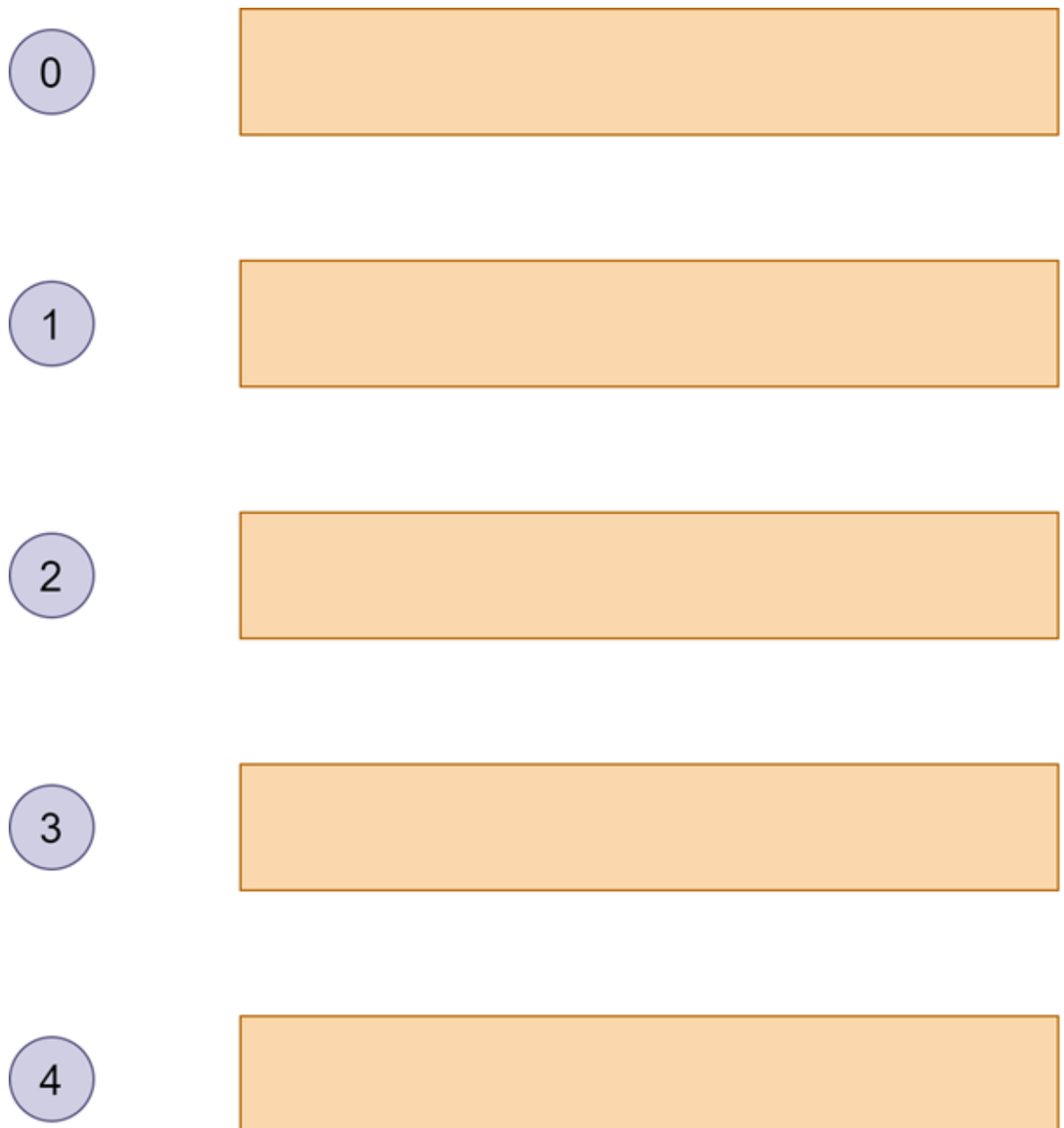
Se retoma el ejemplo de los artistas, ahora insertando las tuplas en una base de datos

Artista.aid	Artista.nombre
1	Leonardo
5	Bernini
8	Michelangelo
11	Brunelleschi
12	Botticelli
16	Giotto

Método: Hash Index

Ejemplo

Artista.aid	Artista.nombre
1	Leonardo
5	Bernini
8	Michelangelo
11	Brunelleschi
12	Botticelli
16	Giotto



Método: Hash Index

Ejemplo

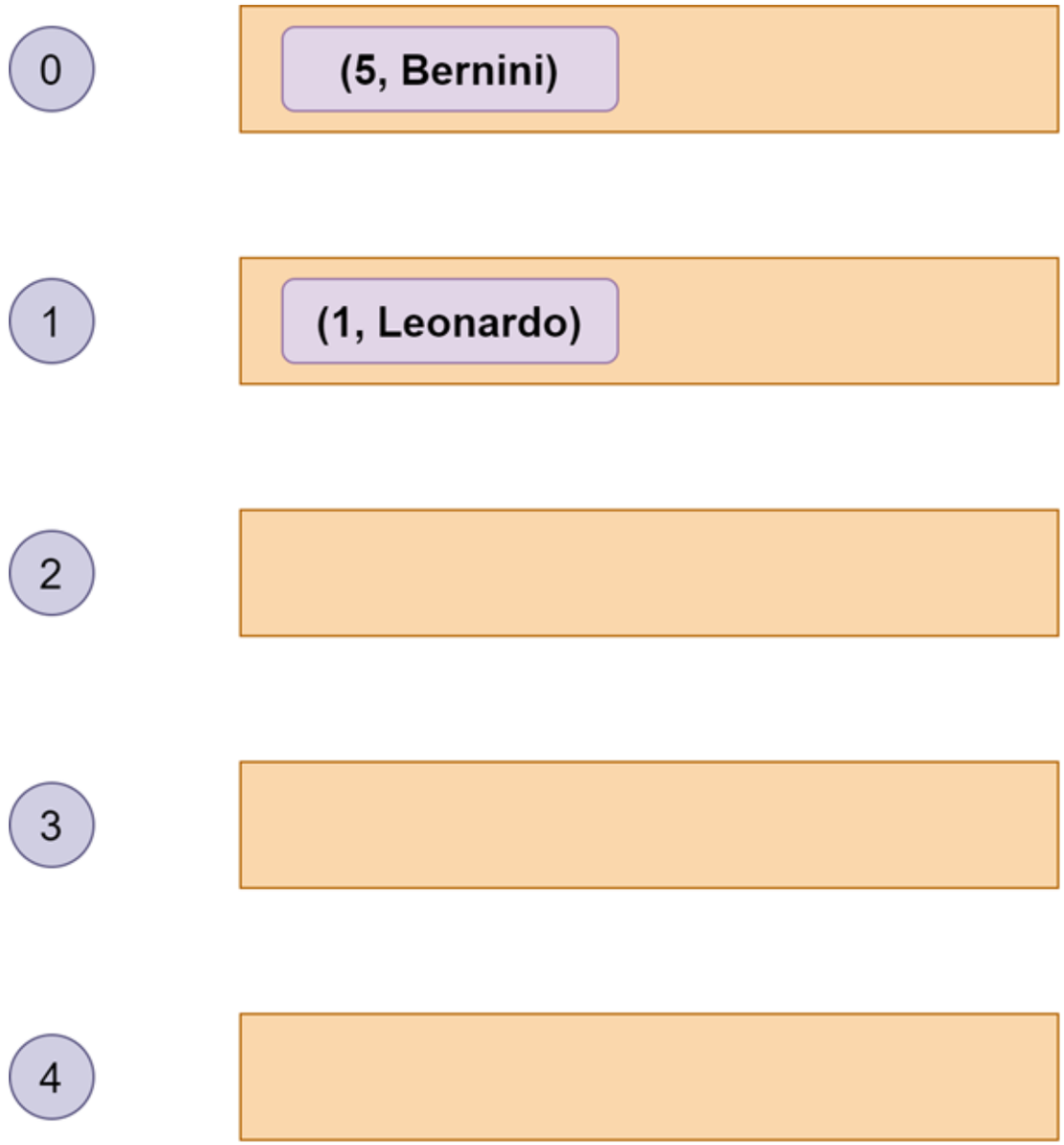
Artista.aid	Artista.nombre
1	Leonardo
5	Bernini
8	Michelangelo
11	Brunelleschi
12	Botticelli
16	Giotto



Método: Hash Index

Ejemplo

Artista.aid	Artista.nombre
1	Leonardo
5	Bernini
8	Michelangelo
11	Brunelleschi
12	Botticelli
16	Giotto



Método: Hash Index

Ejemplo

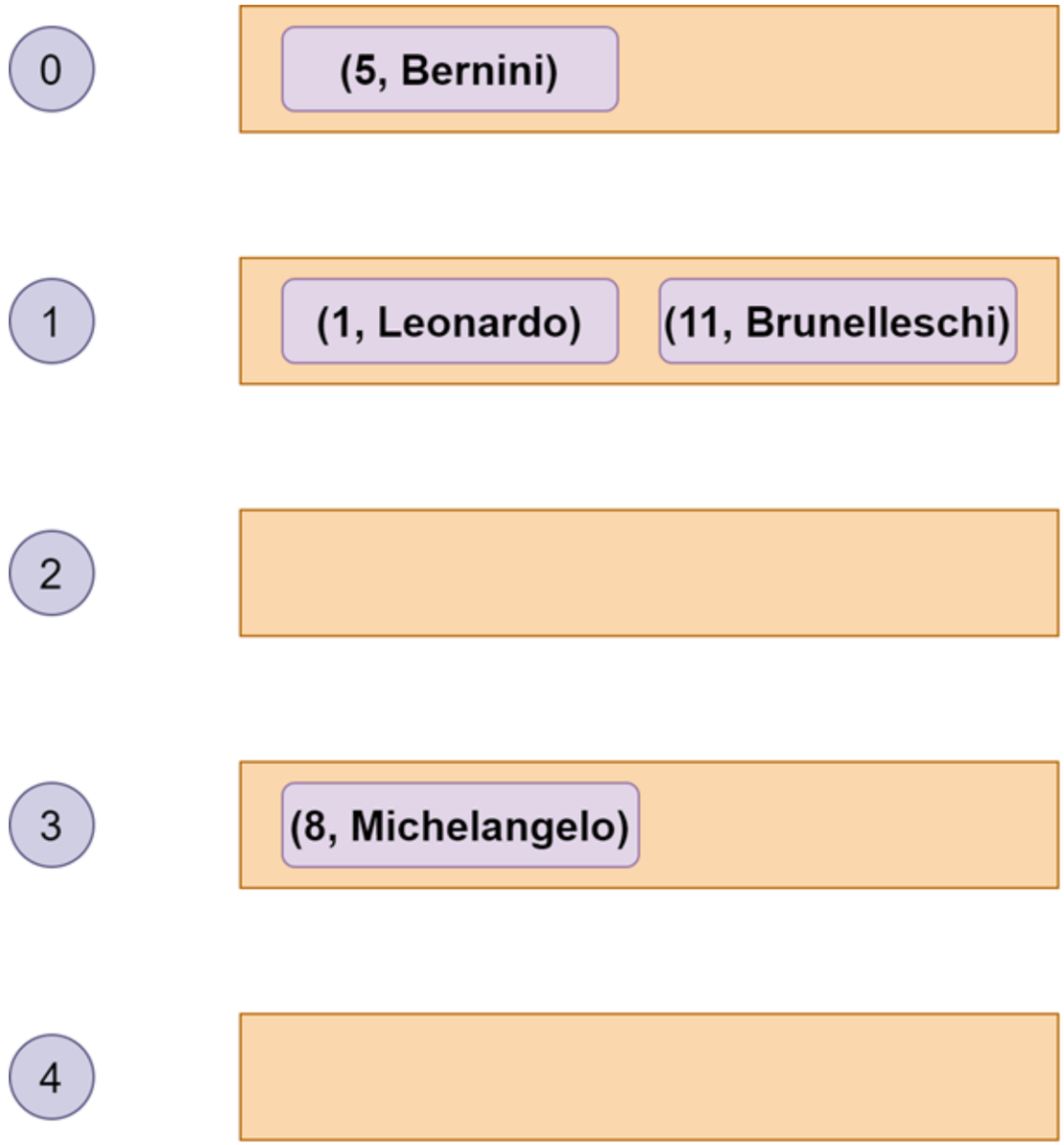
Artista.aid	Artista.nombre
1	Leonardo
5	Bernini
8	Michelangelo
11	Brunelleschi
12	Botticelli
16	Giotto



Método: Hash Index

Ejemplo

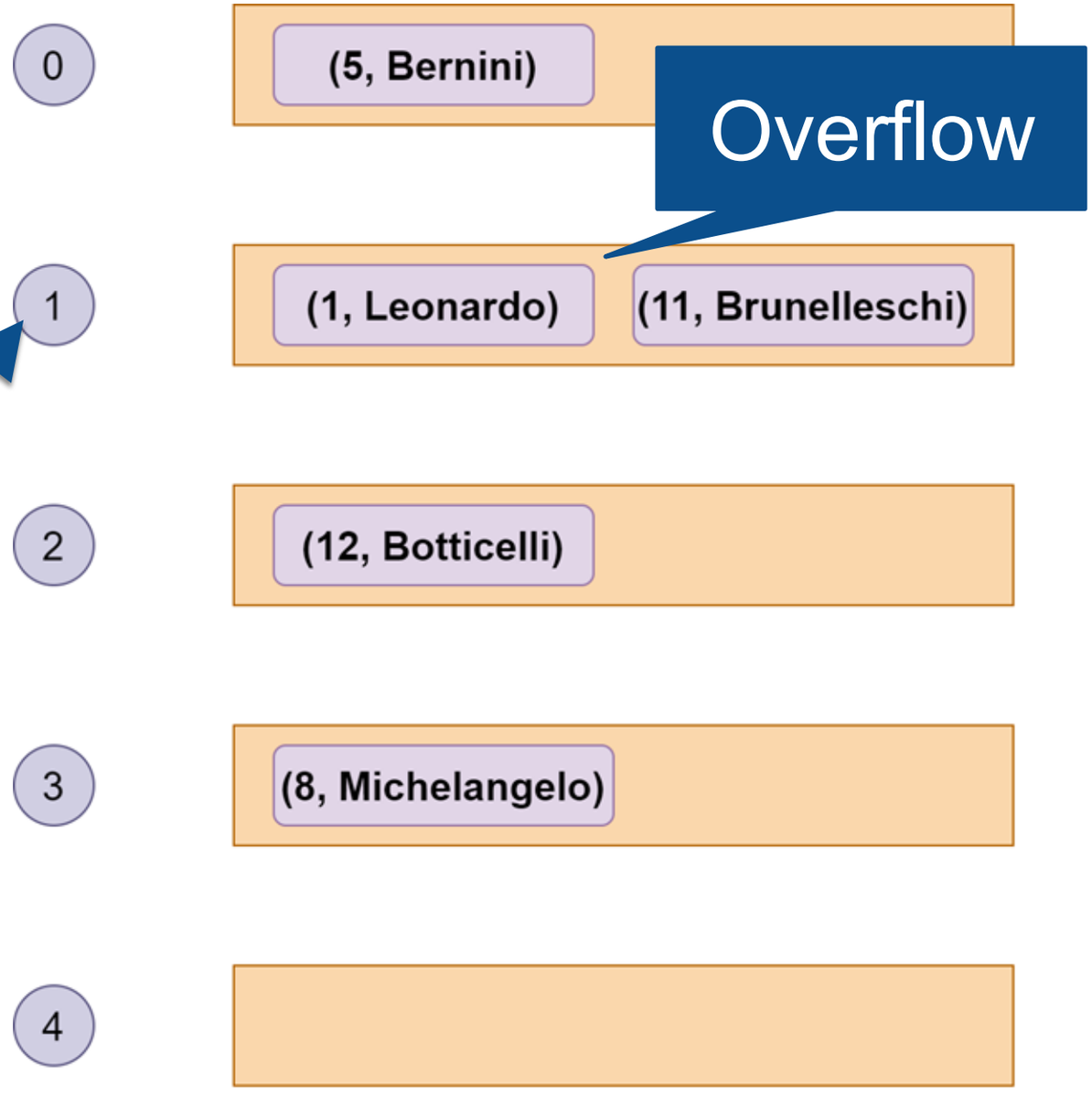
Artista.aid	Artista.nombre
1	Leonardo
5	Bernini
8	Michelangelo
11	Brunelleschi
12	Botticelli
16	Giotto



Método: Hash Index

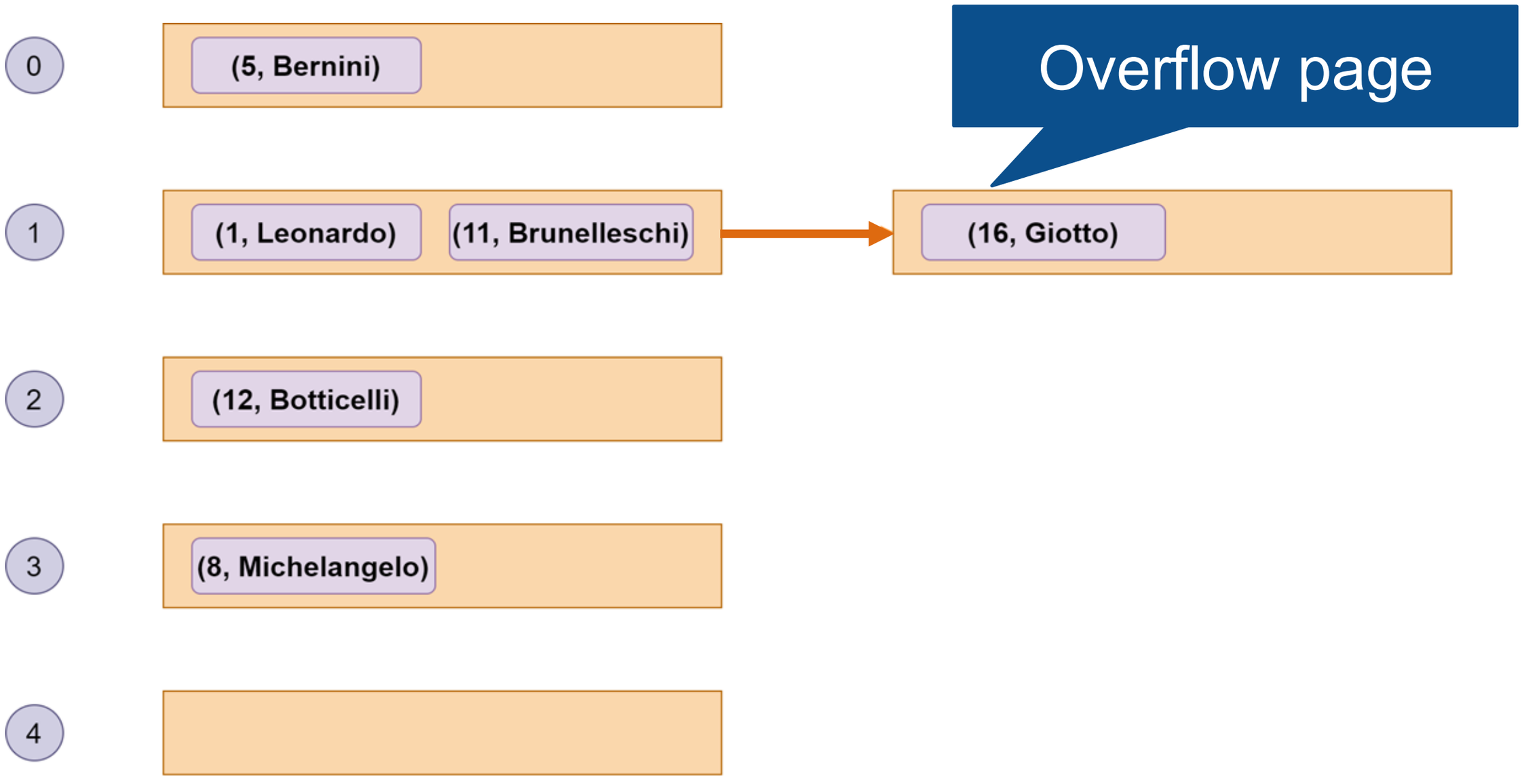
Ejemplo

Artista.aid	Artista.nombre
1	Leonardo
5	Bernini
8	Michelangelo
11	Brunelleschi
12	Botticelli
16	Giotto



Método: Hash Index

Ejemplo



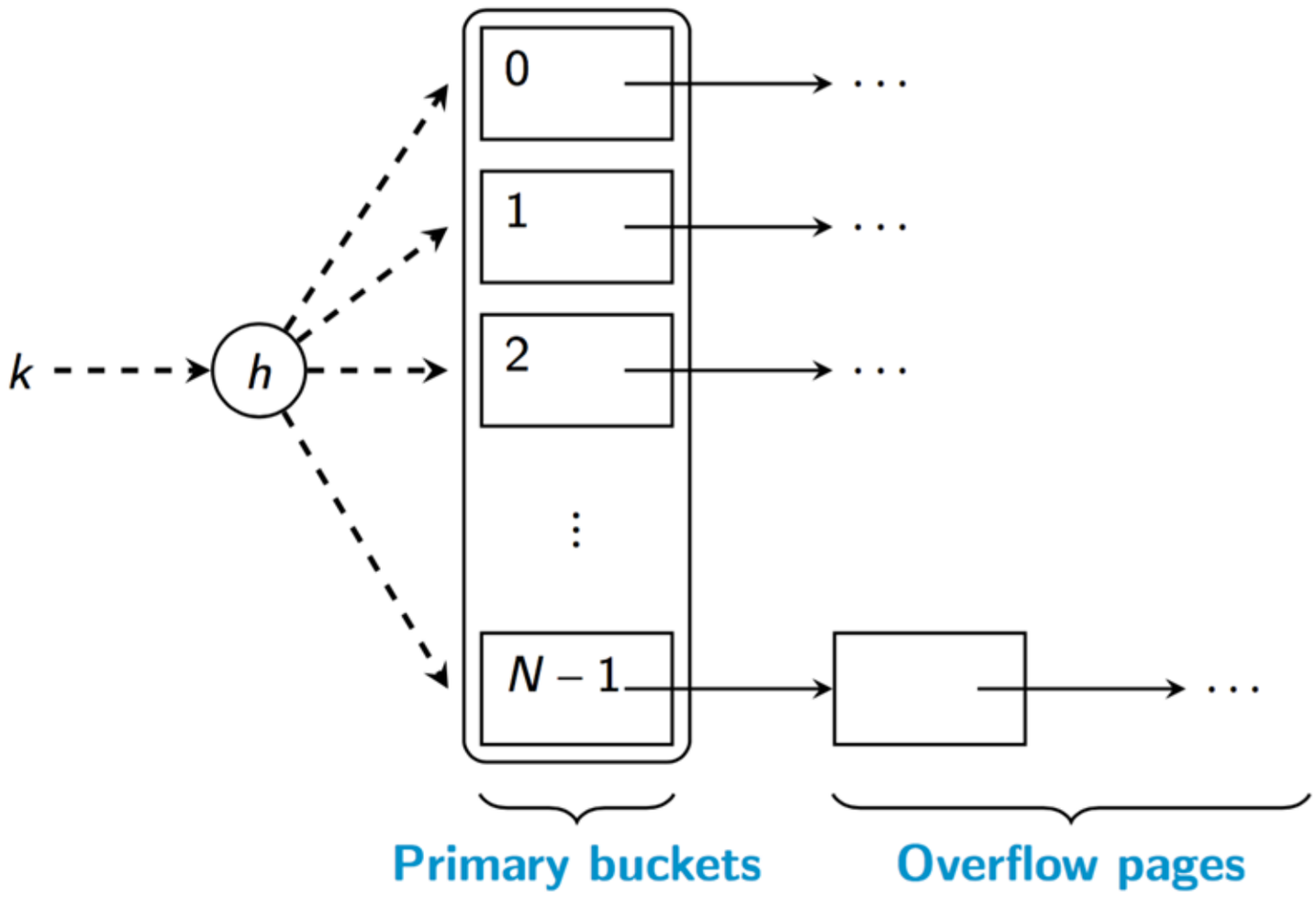
Método: Hash Index

Para una relación **R** y un atributo **A**

- N páginas de disco sirven de **buckets**
- Cada bucket cuenta con una lista ligada de **overflow pages**
- Se usa una función de hash **h** que indica qué bucket corresponde a cada valor:

```
h: dominio(A) → [0, …, N - 1]
```

Método: Hash Index



Método: Hash Index

Para buscar un elemento dada una search key **k**:

- Se computa el valor de **h(k)**
- Se accede al bucket correspondiente
- Se busca el elemento en la página asociada o una de sus overflow pages

costo $O(1)$

Caso ideal: costo 0

Peor caso: costo $O(\lceil \frac{|R|}{B} \rceil)$

¡Cuando todo va a un mismo casillero!

Método: Hash Index

¿Cuándo no conviene utilizar un Hash Index?

```
SELECT * FROM Artistas A WHERE A.aid > 12
```

Para estimar el número de accesos al disco necesarios para esta consulta, se calcula el número de tuplas que satisfacen la condición, dividido por el número de tuplas por bloque:

$$\left\lceil \frac{n}{B} \right\rceil \quad \text{(o también)} \quad \left\lceil \frac{|R|}{B} \right\rceil$$

Aquí, n es el número de registros donde A.aid > 12 y B es la capacidad de registros por bloque.

Los índices hash son ineficientes para consultas que involucran condiciones de rango, como A.aid > 12, ya que están optimizados para búsquedas de igualdad.

Método: B+ Tree Index

Importante: para facilitar inserción de datos nuevos los B+ trees no tienen nodos completamente llenos

Típicamente, las hojas se ocupan solo hasta cierto porcentaje; e.g., 60% significa que, si caben 100 tuplas, hay solo 60 en una página de la hoja

Método: B+ Tree Index

El costo de búsqueda es logarítmico $\sim log(|R|)$

Si **B** es el tamaño de una página en tuplas, el costo de búsqueda es:

$$O(log_B(\left\lceil \frac{|R|}{B} \right\rceil))$$

¿Qué ocurre si el índice se mantiene en memoria RAM?

Tipos de valores a guardar

Un índice puede retornar:

- *La tupla* buscada
- Un *puntero* o lista de punteros a las páginas del disco duro que contienen las tuplas buscadas

Clustered y Unclustered Index

Data entry:
La tupla misma

Se habla de un índice **Clustered** si, al consultarlo por un valor, retorna una tupla

Se habla de un índice **Unclustered** si retorna un puntero

Data entry:
Puntero a la página
con la tupla

Clustered y Unclustered Index

Clustered Index se conoce como índice primario

Unclustered Index se conoce como índice secundario

Clustered y Unclustered Index

Considere Empleados(id, nombre, edad)

Un índice primario en general se aplica sobre la primary key, por ejemplo el id de los empleados

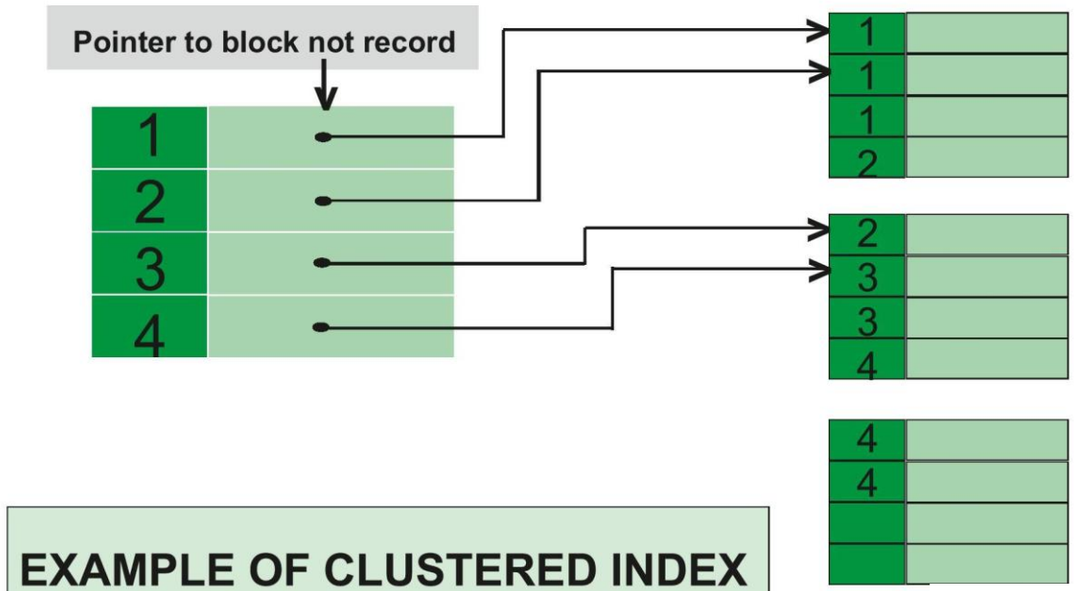
Un índice secundario se aplica sobre un atributo como edad (para ordenar por edad con mayor rapidez)



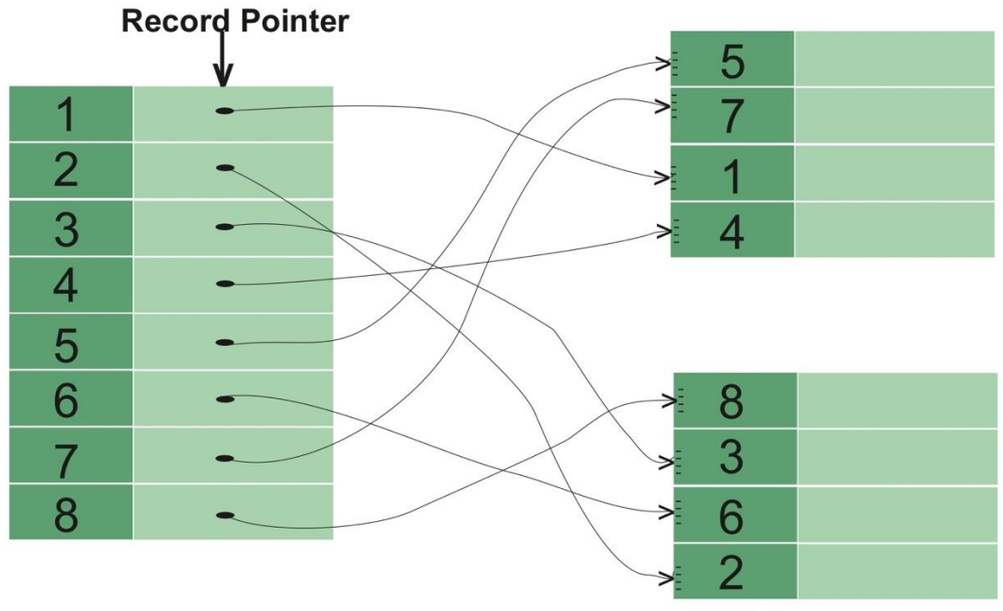
Clustered / Unclustered

Buscando una tupla con distintos tipos de índices

Índice clustered y unclustered: ejemplo



EXAMPLE OF CLUSTERED INDEX



EXAMPLE OF NON-CLUSTERED INDEX

Hash Index clustered

R(a,b,c) … índice sobre a

a = 5

5 % 4 = 1

Bucket	
0	(4,1,1), (0,2,3), (8,1,2)
1	(1,2,2), (5,1,1)
2	(2,3,3)
3	(3,1,2), (7,8,9)

Hash Index clustered

R(a,b,c) … índice sobre a

a = 5

5 % 4 = 1

Bucket	
0	(4,1,1), (0,2,3), (8,1,2)
1	(1,2,2), (5,1,1) página del disco
2	(2,3,3)
3	(3,1,2), (7,8,9)

Hash Index clustered

R(a,b,c) … índice sobre a

a = 5

5 % 4 = 1

Bucket	
0	(4,1,1), (0,2,3), (8,1,2)
1	(1,2,2), (5,1,1) página del disco
2	(2,3,3)
3	(3,1,2), (7,8,9)

#(I/O) = 1 (el bucket es la página con la tupla)

Hash Index unclustered

R(a,b,c) … índice sobre a

a = 5

5 % 4 = 1

Bucket	
0	(4,p1), (0,p2), (8,p3)
1	(1,p1), (5,p1)
2	(2,p2)
3	(3,p2), (7,p3)

Datos en el disco:

p1	p2	p3
(4,1,1), (1,2,2), (5,1,1)	(0,2,3), (3,1,2), (2,3,3)	(7,8,9), (8,1,2)

Hash Index unclustered

R(a,b,c) … índice sobre a

a = 5

5 % 4 = 1

Bucket	
0	(4,p1), (0,p2), (8,p3)
1	(1,p1), (5,p1) página del disco
2	(2,p2)
3	(3,p2), (7,p3)

Datos en el disco:

p1	p2	p3
(4,1,1), (1,2,2), (5,1,1)	(0,2,3), (3,1,2), (2,3,3)	(7,8,9), (8,1,2)

Hash Index unclustered

R(a,b,c) … índice sobre a

a = 5

5 % 4 = 1

Bucket	
0	(4,p1), (0,p2), (8,p3)
1	(1,p1), (5,p1) página del disco
2	(2,p2)
3	(3,p2), (7,p3)

Datos en el disco:

p1

(4,1,1), (1,2,2), (5,1,1)

página del disco

p2

(0,2,3), (3,1,2), (2,3,3)

p3

(7,8,9), (8,1,2)

Hash Index unclustered

R(a,b,c) … índice sobre a

a = 5

5 % 4 = 1

Bucket	
0	(4,p1), (0,p2), (8,p3)
1	(1,p1), (5,p1) página del disco
2	(2,p2)
3	(3,p2), (7,p3)

Datos en el disco:

p1

(4,1,1), (1,2,2), (5,1,1)

página del disco

p2

(0,2,3), (3,1,2), (2,3,3)

p3

(7,8,9), (8,1,2)

#(I/O) = 2

Hash Index unclustered

R(a,b,c) … índice sobre a

Considerar: página del índice vs página con tuplas

P punteros/página

(1,p1), (2,p7),…

B tuplas/página

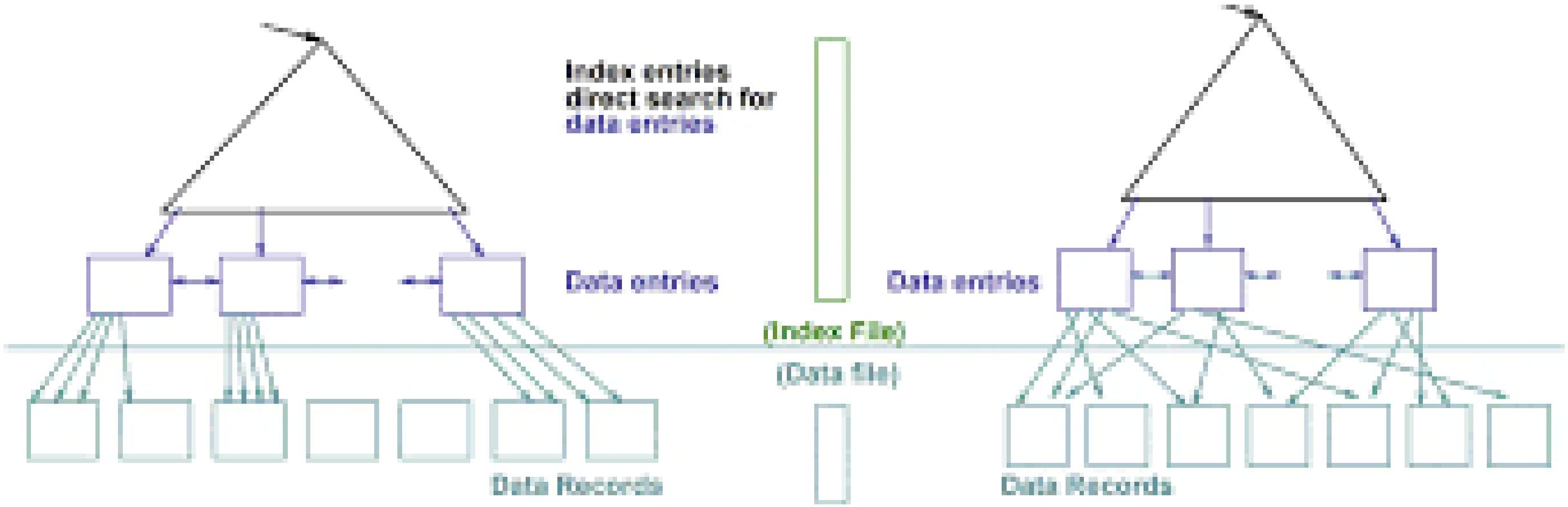
(1,2,3), (7,3,1),…

puntero p1 … un long/8 bytes

tupla … puede ser 20 longs/160 bytes

¿Dónde cabe más? (P>B)

Índice clustered y nonclustered: estructura

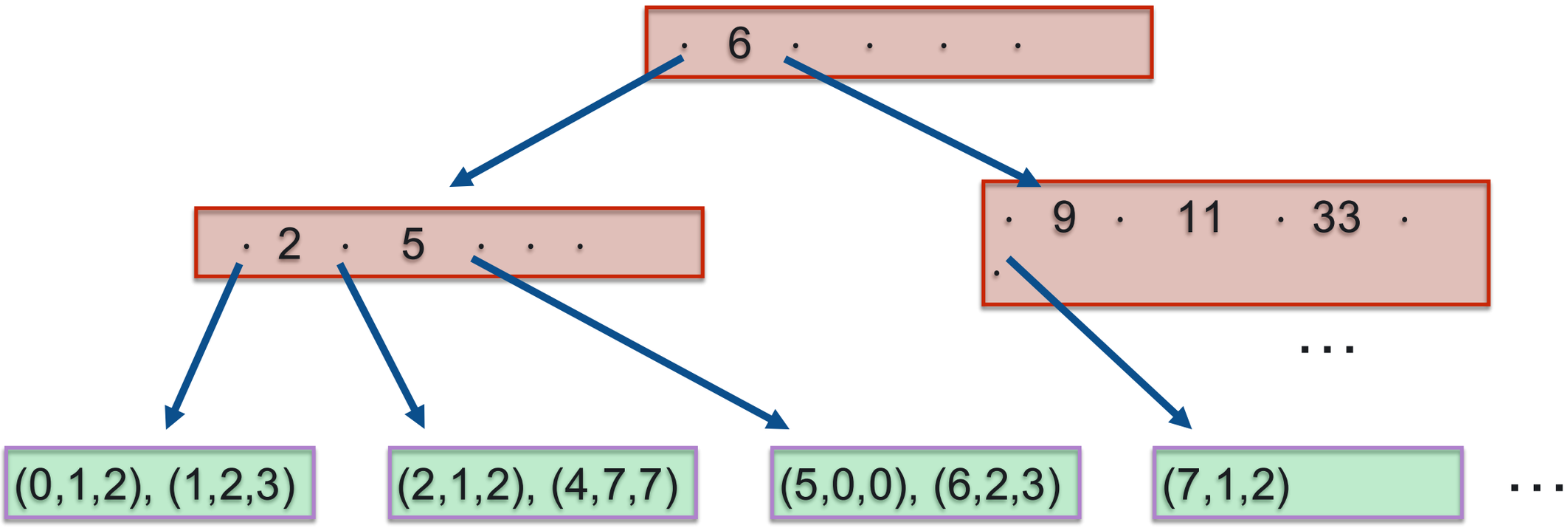


Clustered

Nonclustered

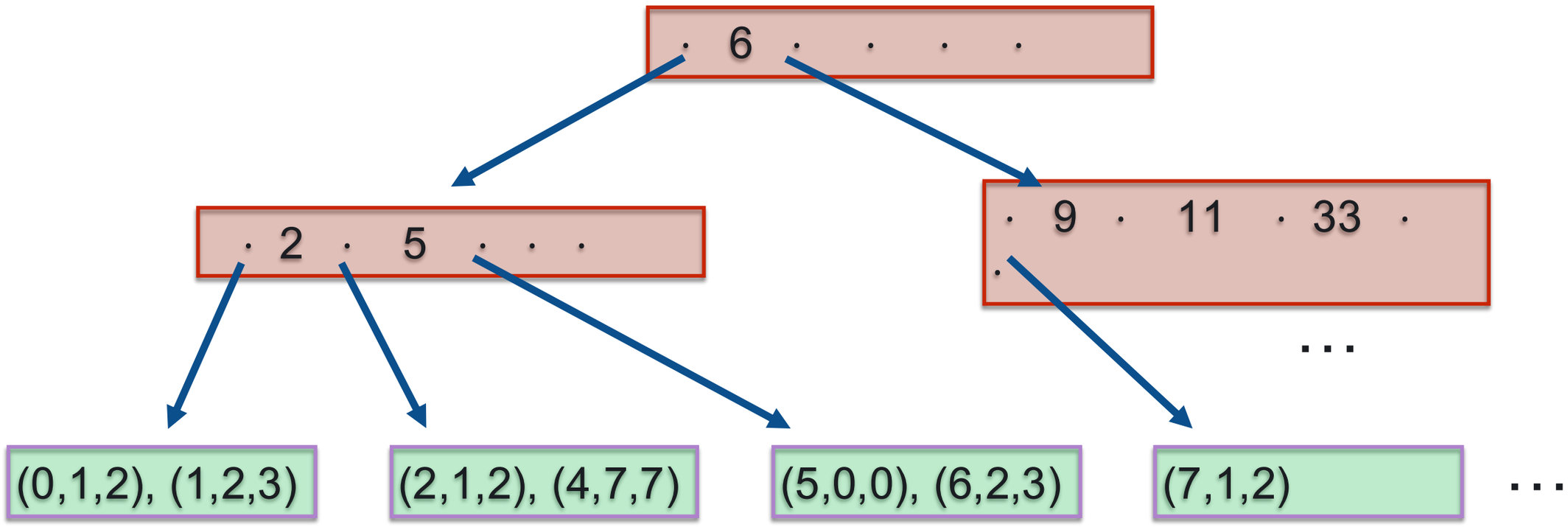
B+ tree clustered

R(a,b,c) … índice sobre a



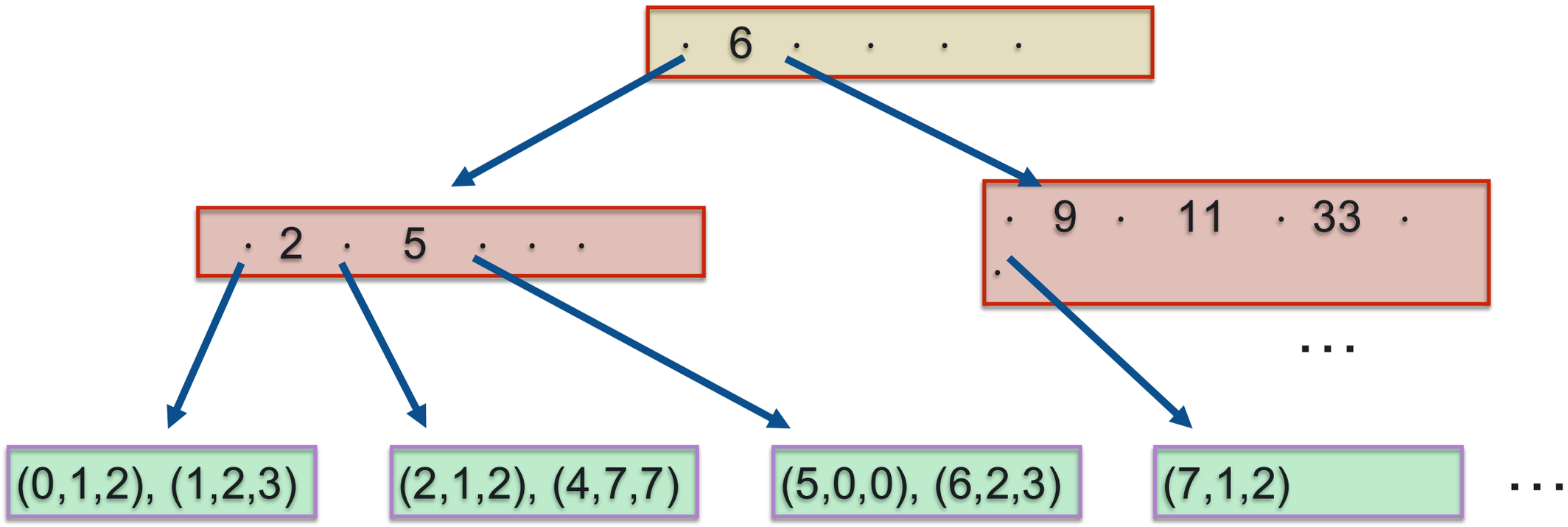
B+ tree clustered

R(a,b,c) … índice sobre a … a = 5



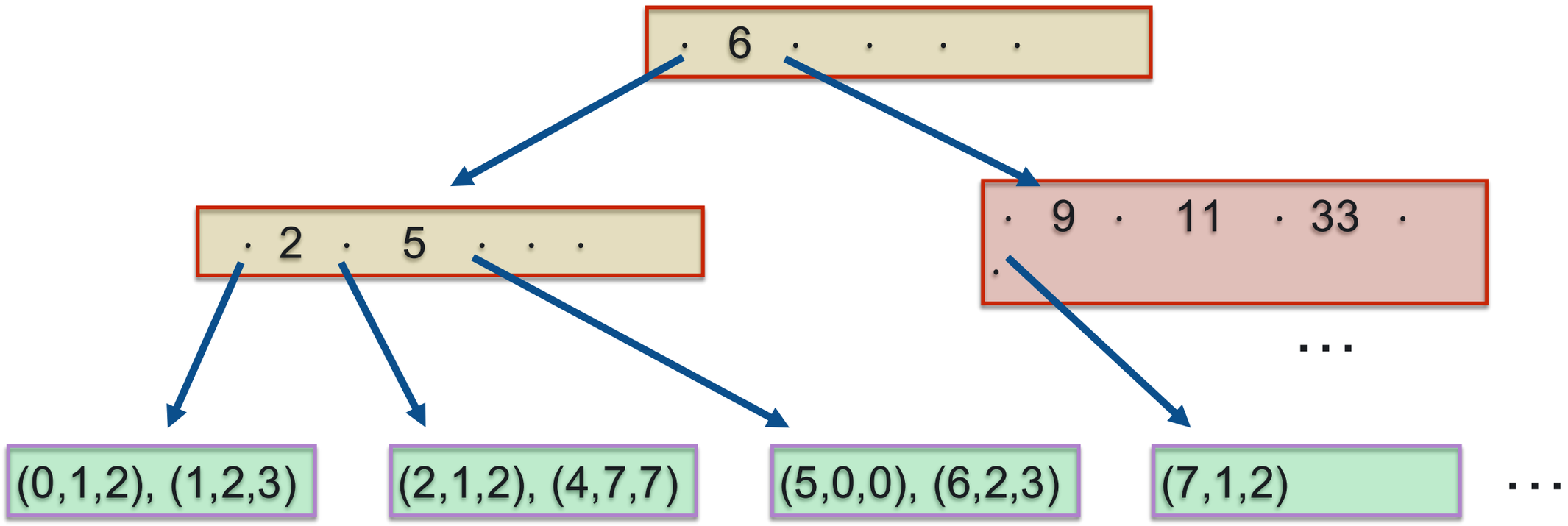
B+ tree clustered

R(a,b,c) … índice sobre a … a = 5



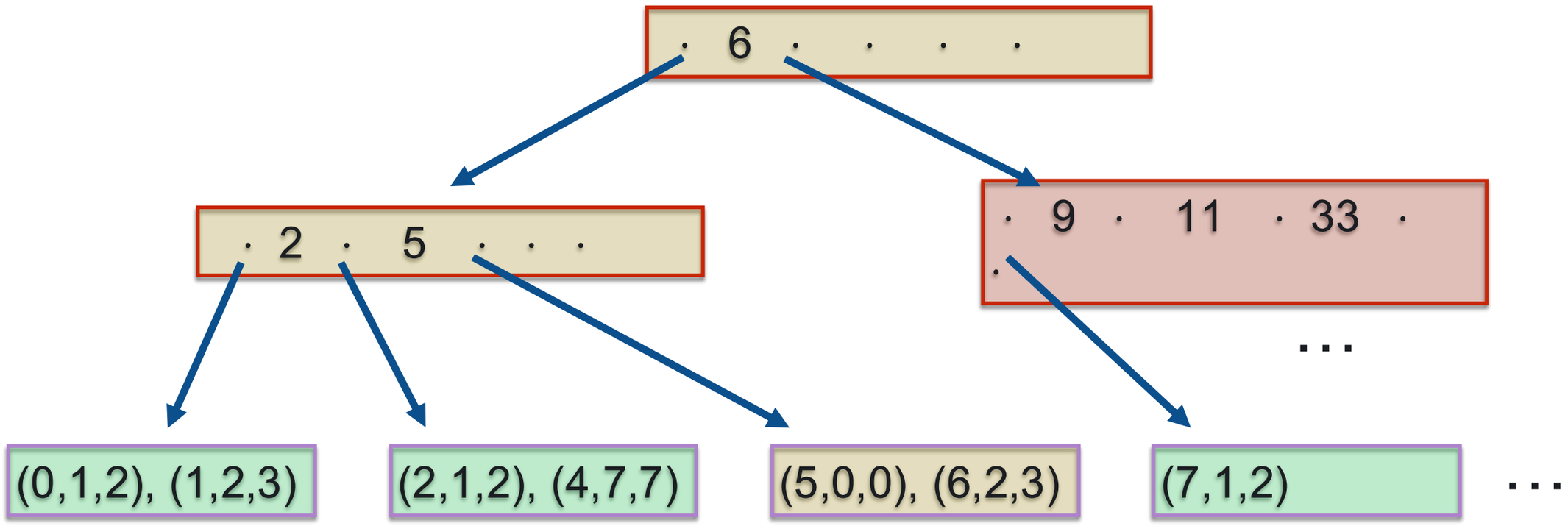
B+ tree clustered

R(a,b,c) … índice sobre a … a = 5



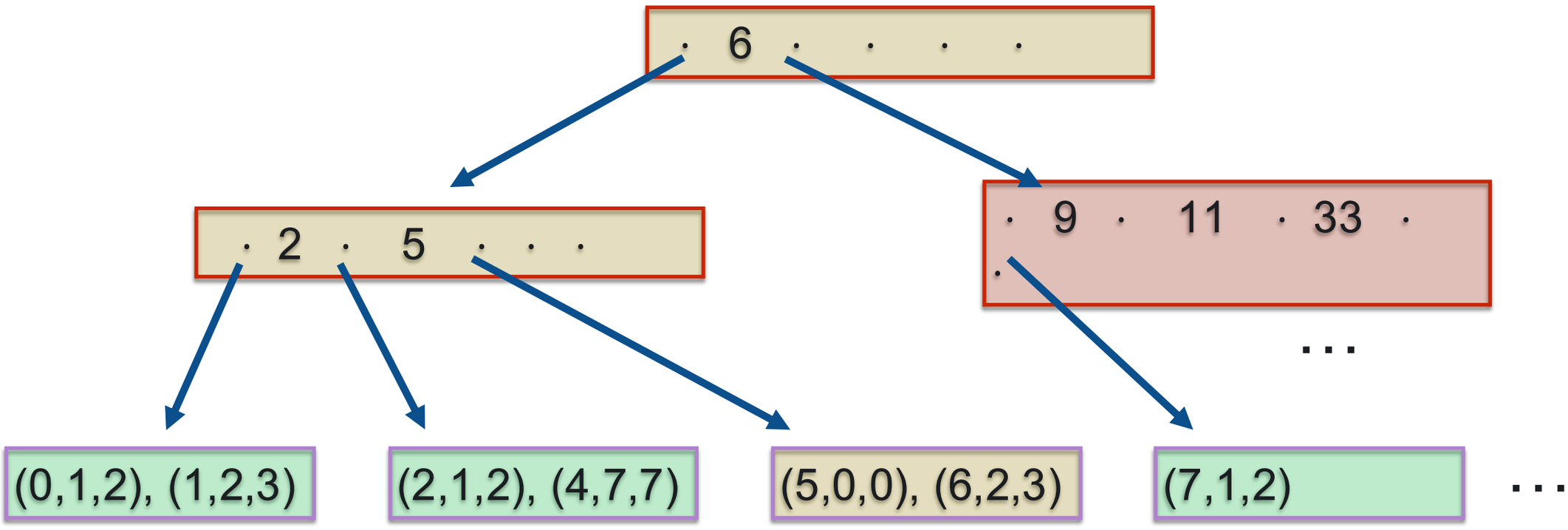
B+ tree clustered

R(a,b,c) … índice sobre a … a = 5



B+ tree clustered

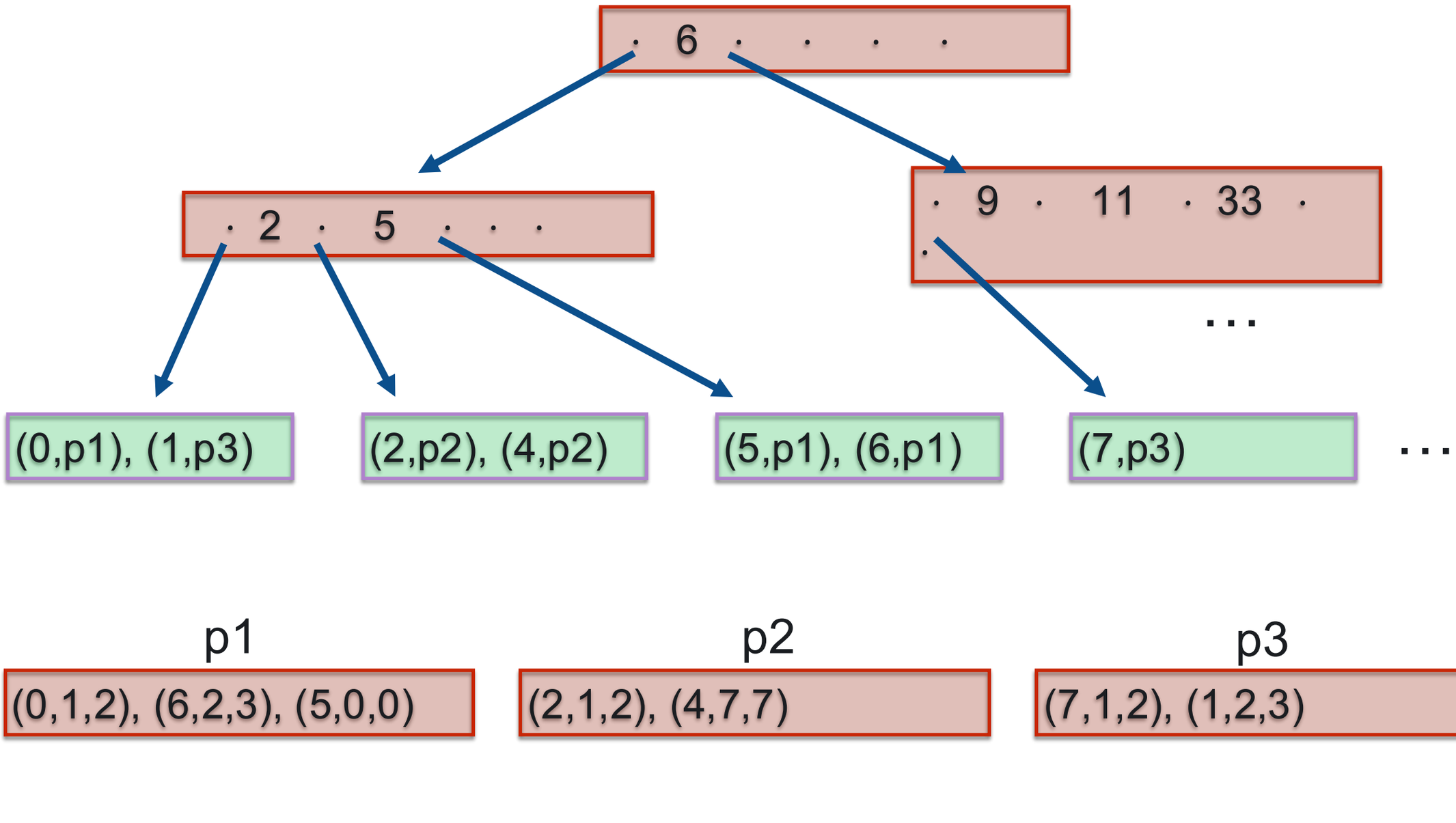
R(a,b,c) … índice sobre a … a = 5



#(I/O) = h … altura del árbol (los datos están en hojas)

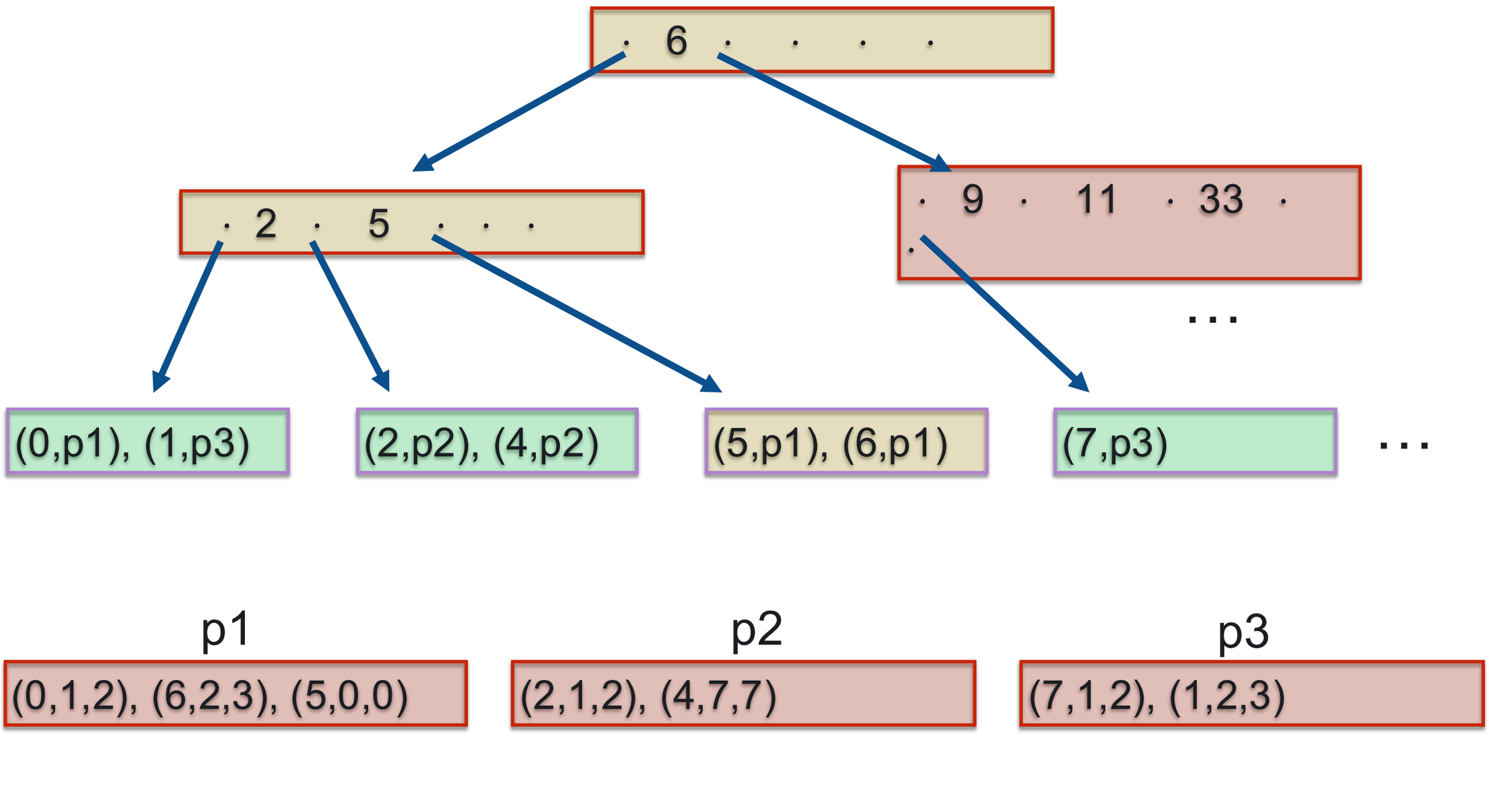
B+ tree unclustered

R(a,b,c) … índice sobre a



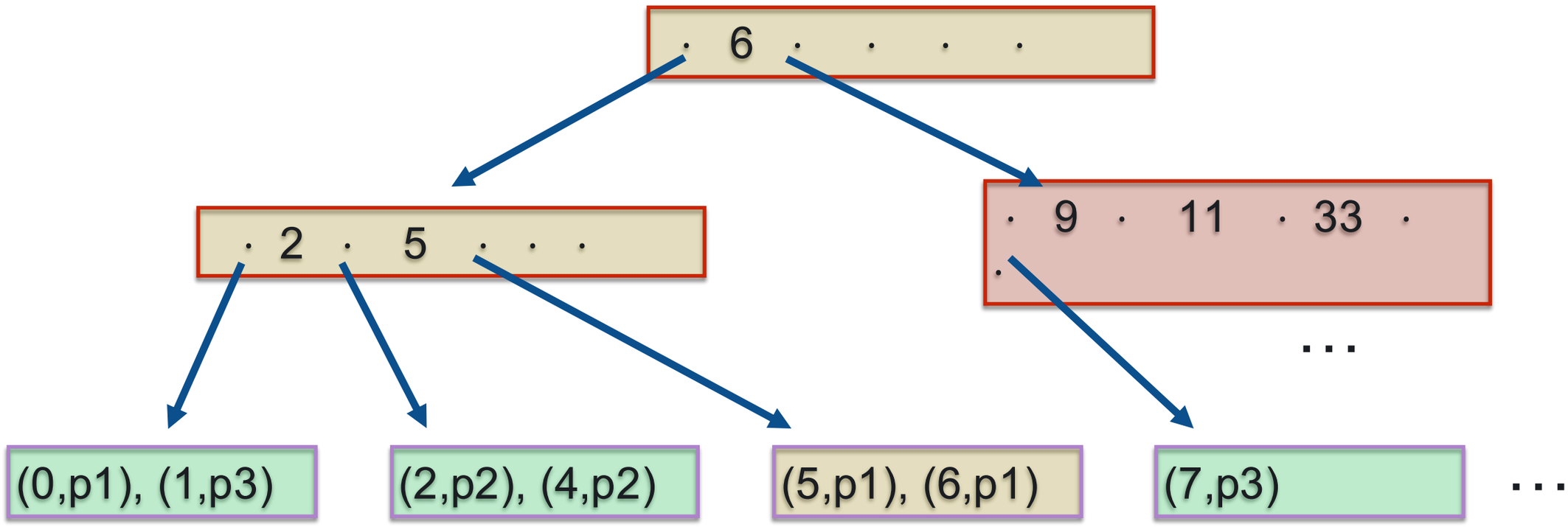
B+ tree unclustered

R(a,b,c) … índice sobre a … a = 5



B+ tree unclustered

R(a,b,c) … índice sobre a … a = 5



p1

(0,1,2), (6,2,3), (5,0,0)

p2

(2,1,2), (4,7,7)

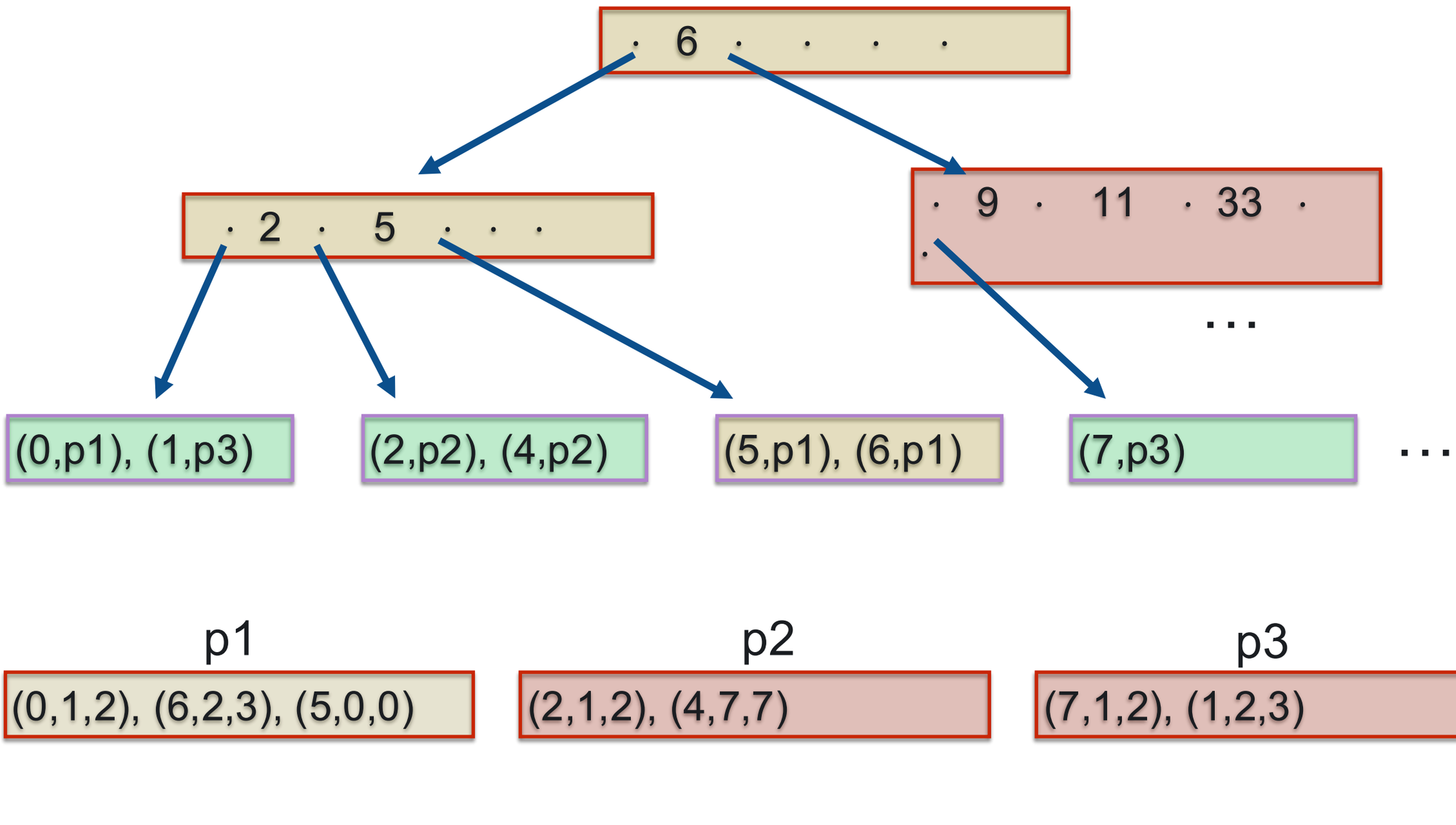
p3

(7,1,2), (1,2,3)

B+ tree unclustered

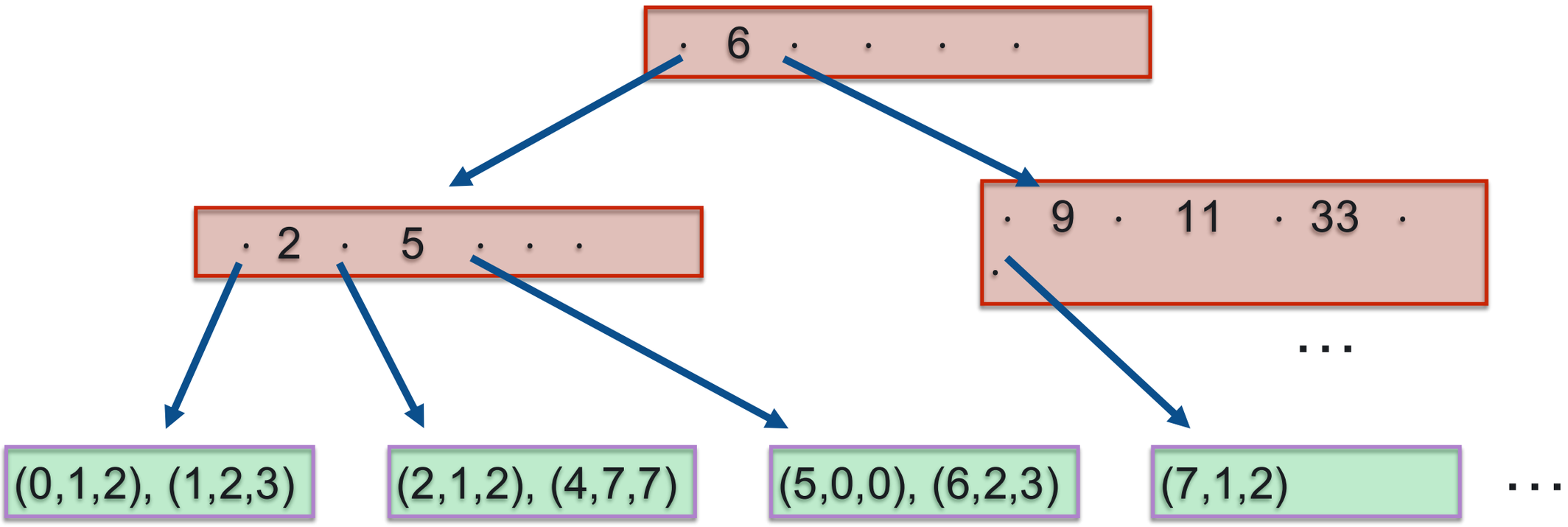
R(a,b,c) … índice sobre a … a = 5

Costo *h+1*



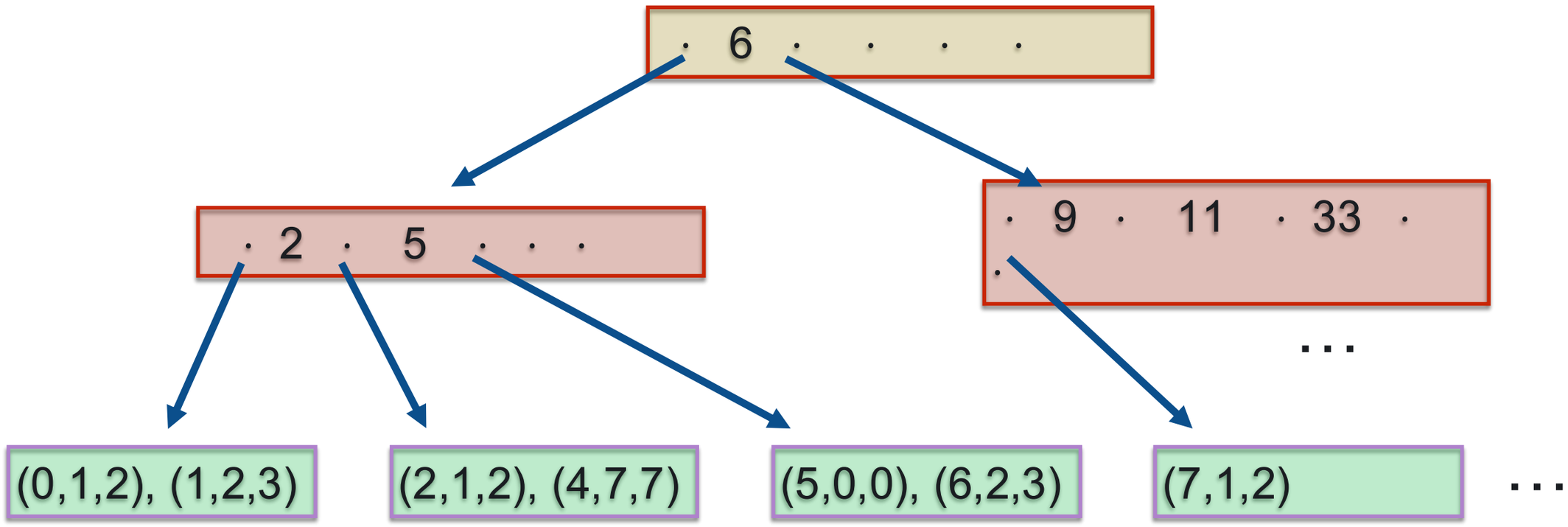
B+ tree clustered

R(a,b,c) … índice sobre a … a > 5



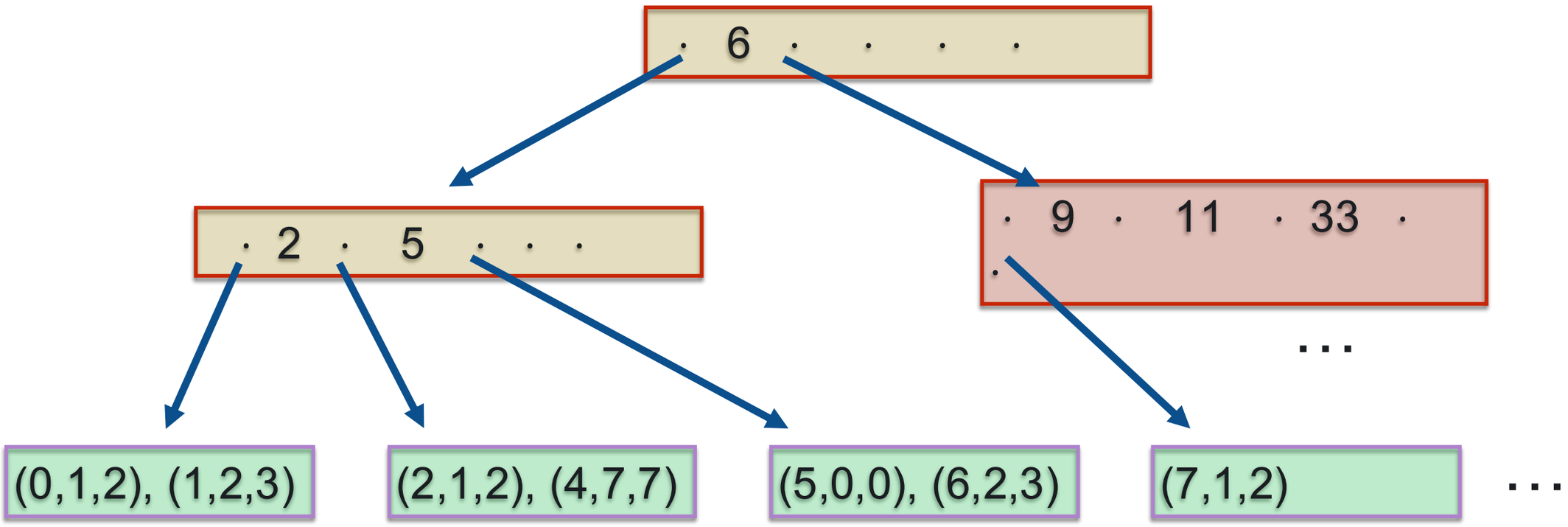
B+ tree clustered

R(a,b,c) … índice sobre a … a > 5



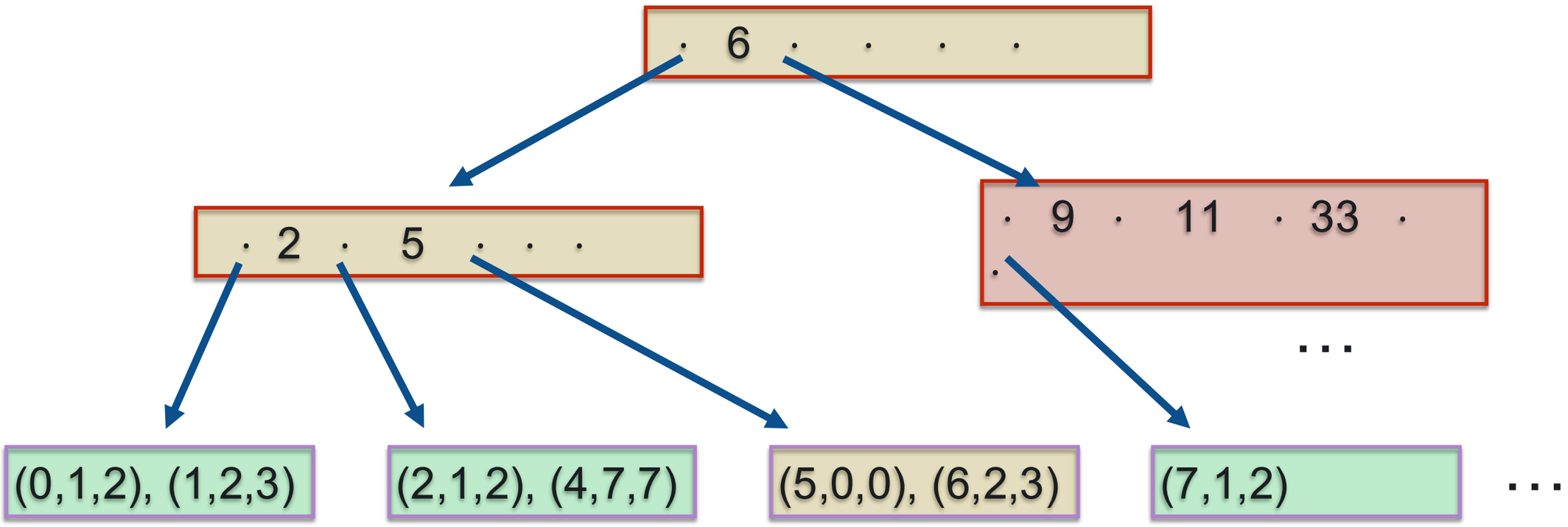
B+ tree clustered

R(a,b,c) … índice sobre a … a > 5



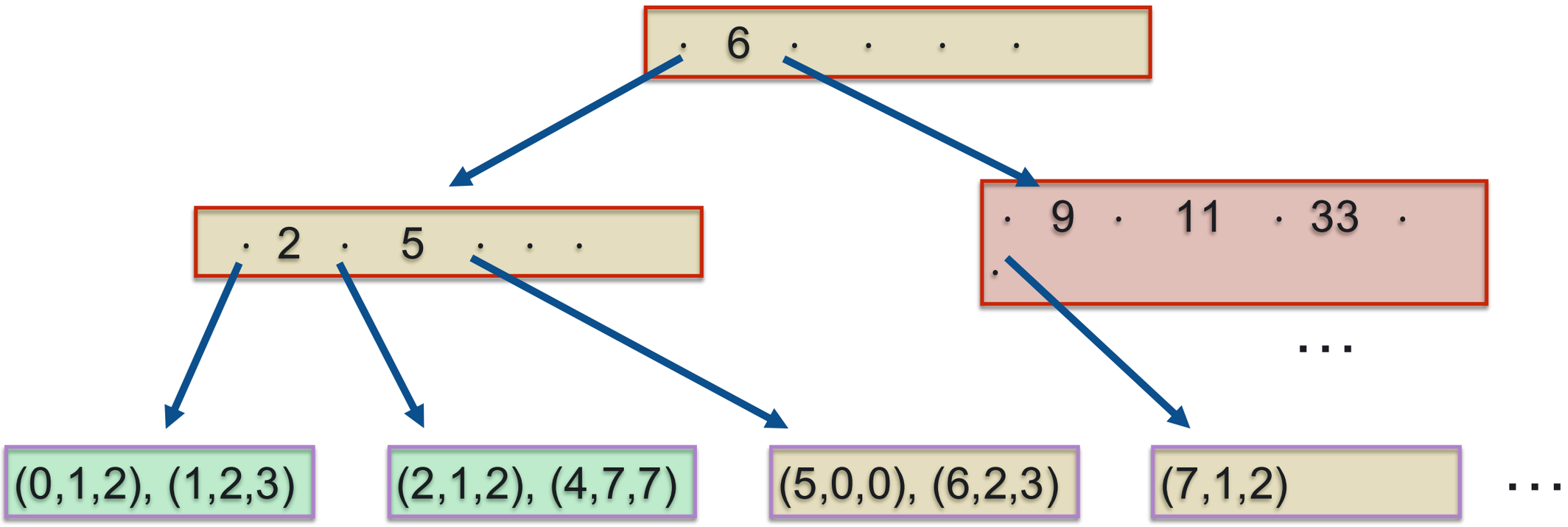
B+ tree clustered

R(a,b,c) … índice sobre a … a > 5



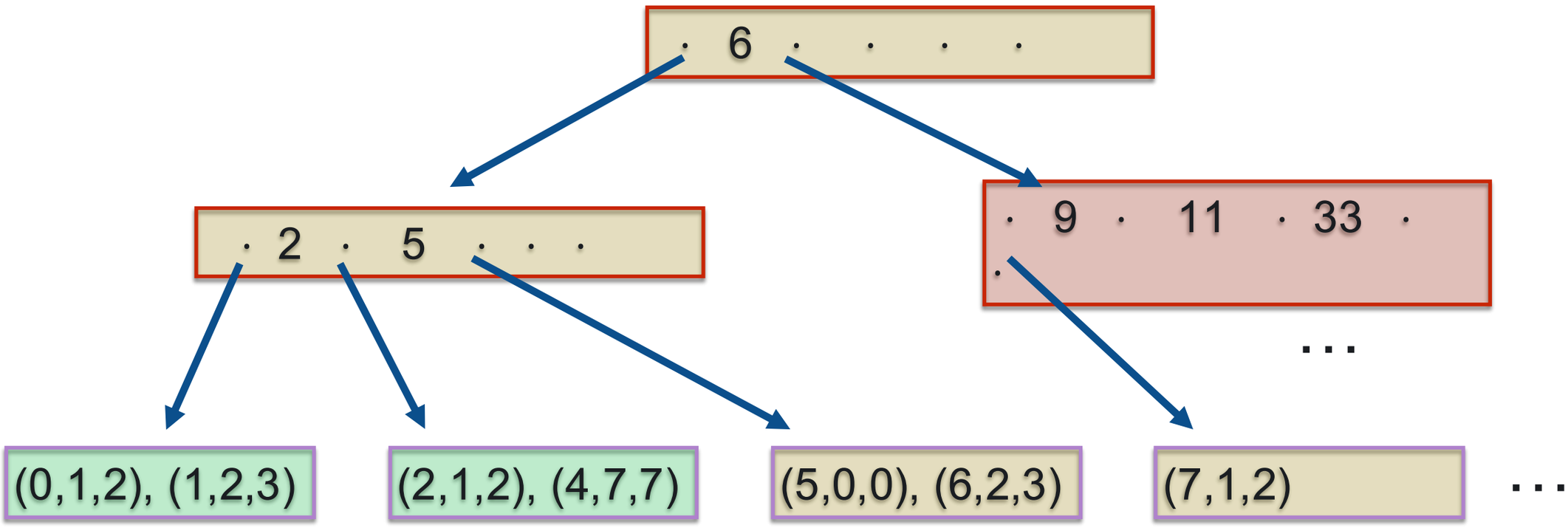
B+ tree clustered

R(a,b,c) … índice sobre a … a > 5



B+ tree clustered

R(a,b,c) … índice sobre a … a > 5

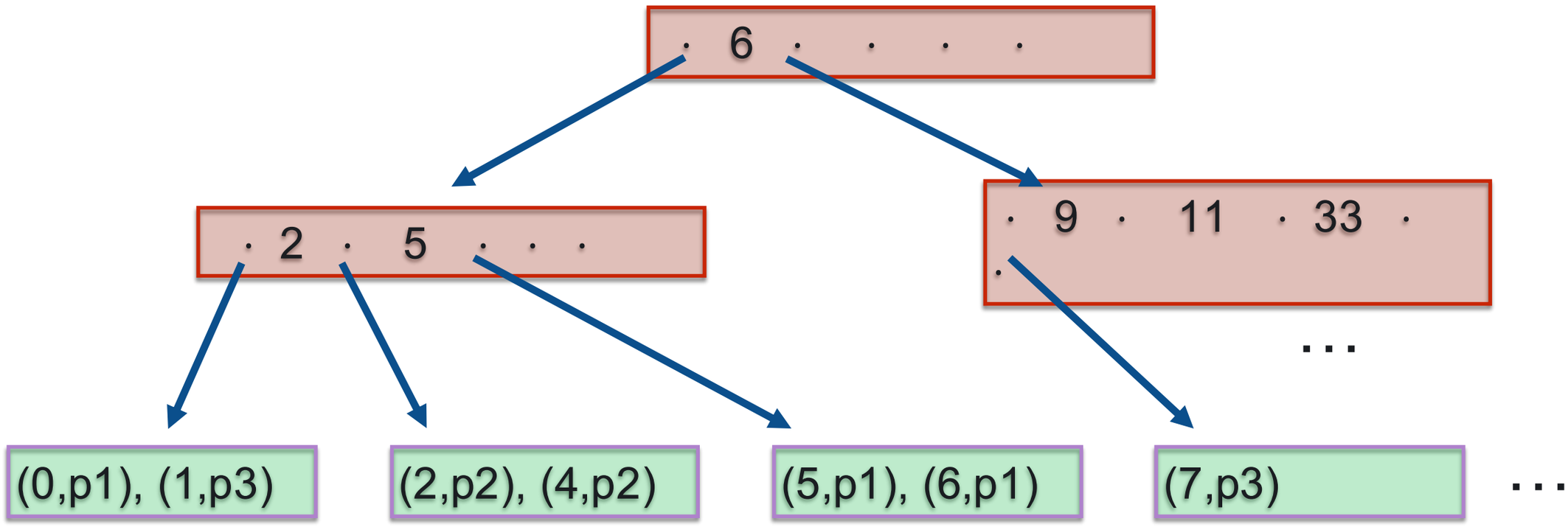


$$\#(\text{I/O}) = h + B_m$$

B_m: nr de bloques con tuplas que calzan búsqueda

B+ tree unclustered

R(a,b,c) … índice sobre a … a > 5



p1

(0,1,2), (6,2,3), (5,0,0)

p2

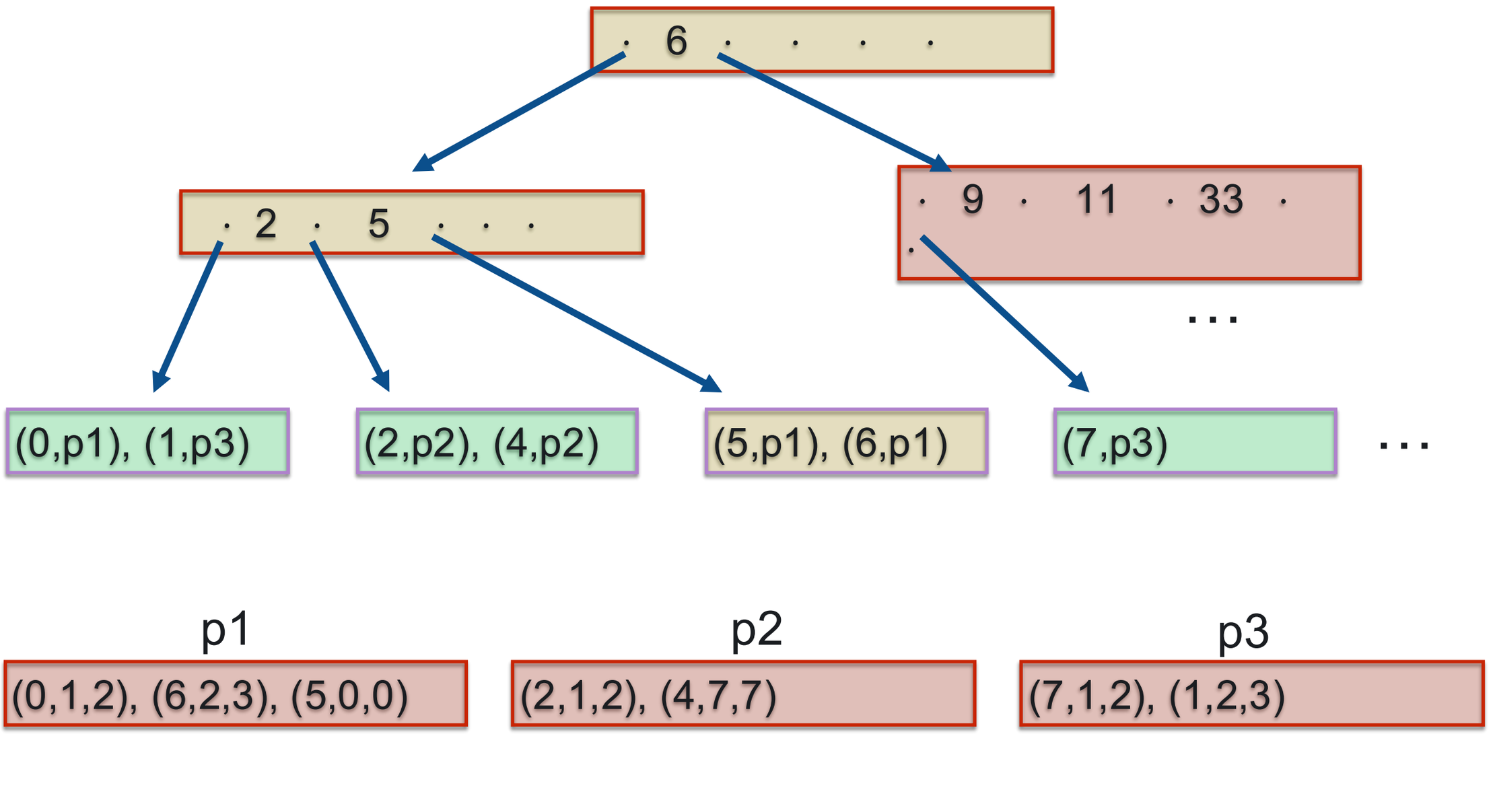
(2,1,2), (4,7,7)

p3

(7,1,2), (1,2,3)

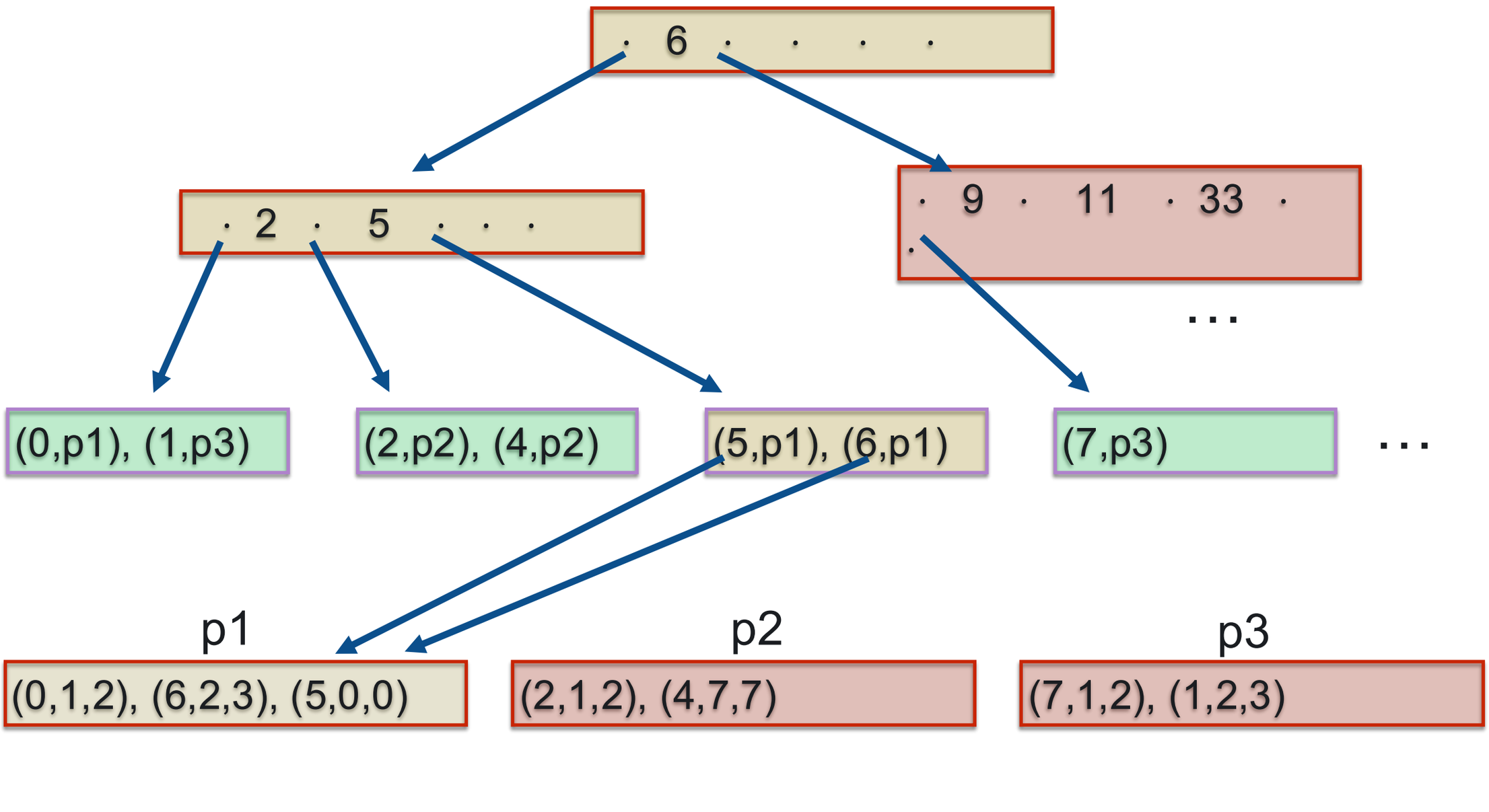
B+ tree unclustered

R(a,b,c) … índice sobre a … a > 5



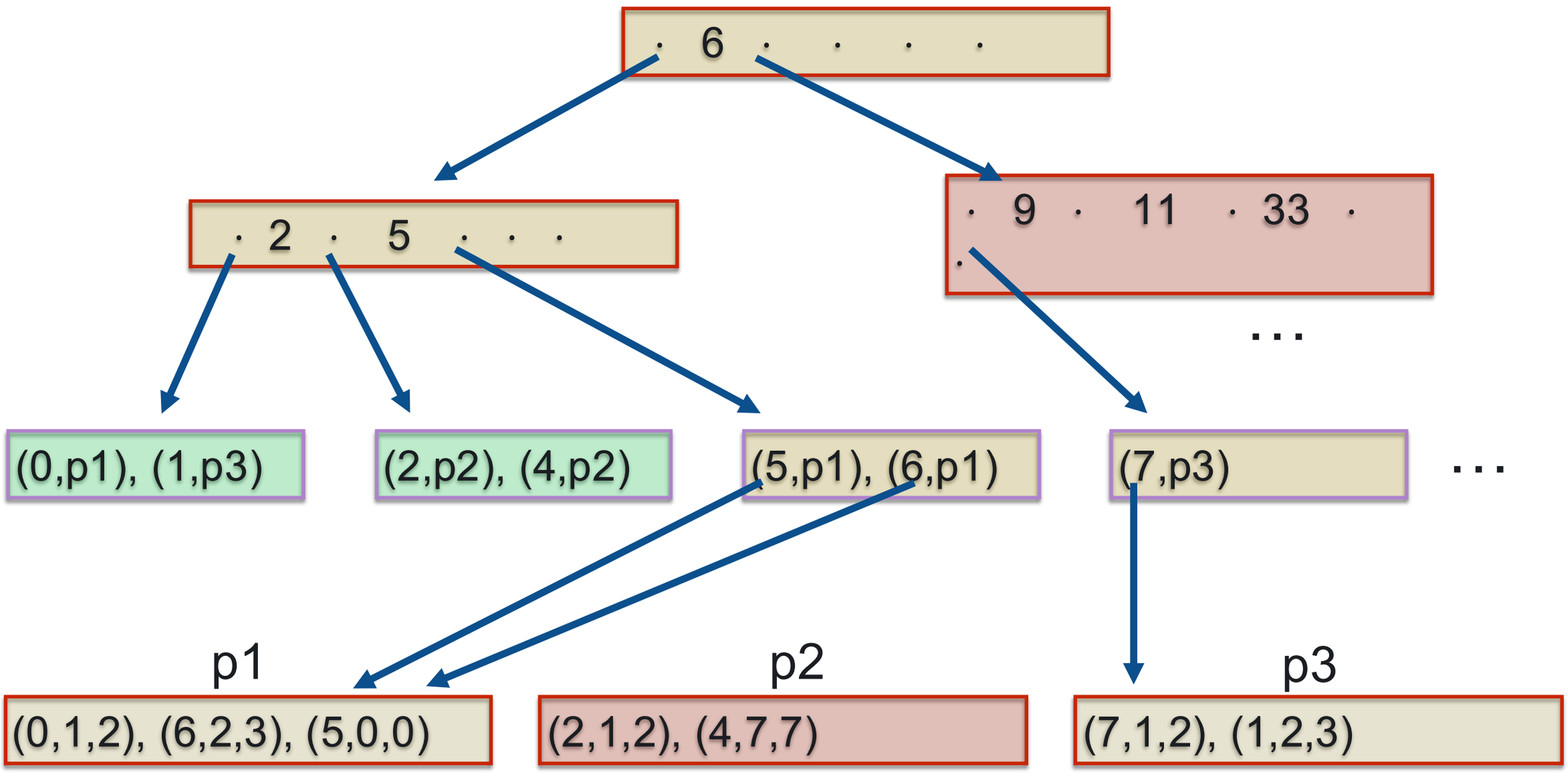
B+ tree unclustered

R(a,b,c) … índice sobre a … a > 5



B+ tree unclustered

R(a,b,c) … índice sobre a … a > 5

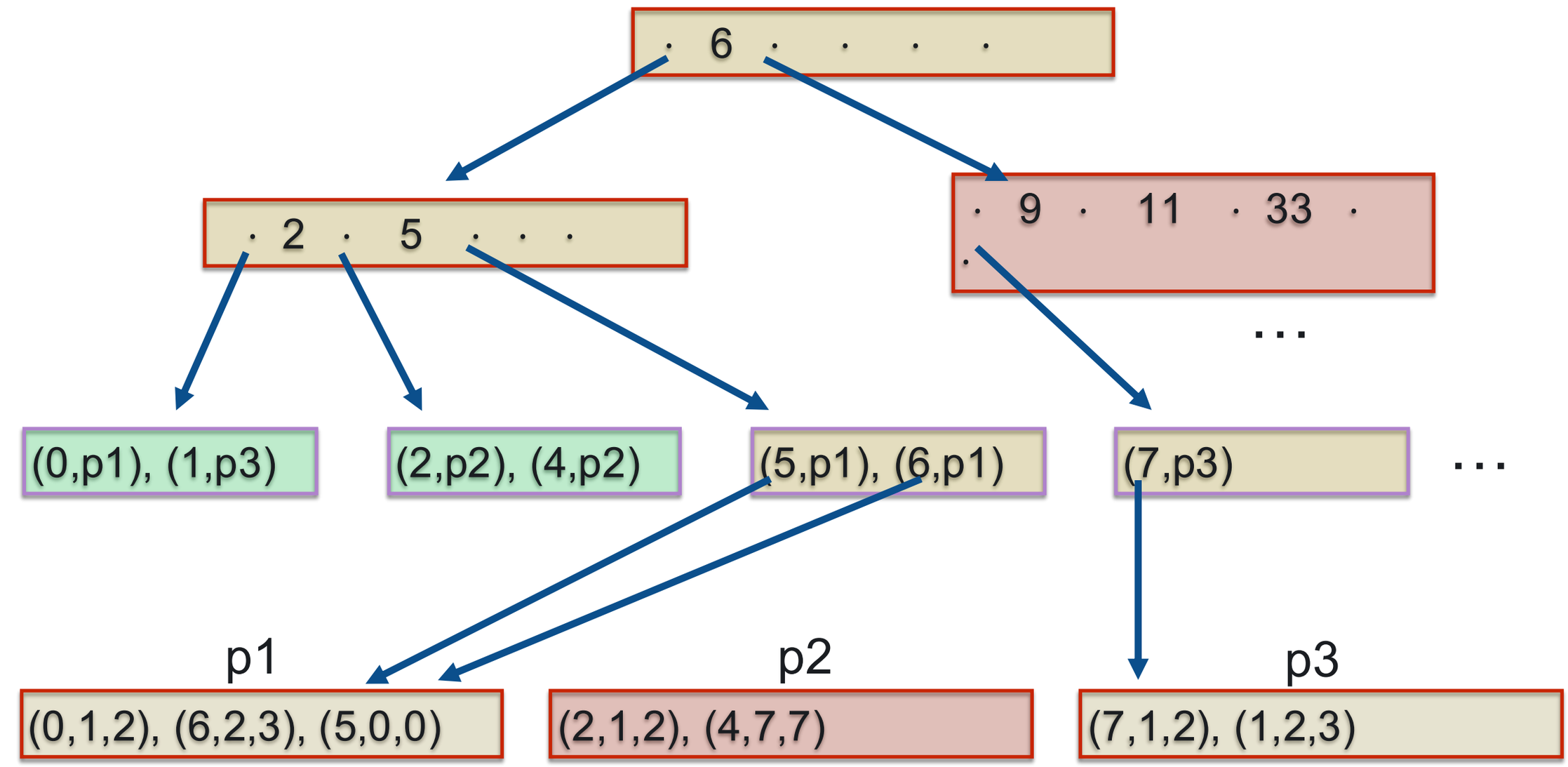


B+ tree unclustered

$$\#(I/O) = h + \left\lceil \frac{n}{P} \right\rceil + n$$

n : número de tuplas que calzan con búsquedas
P : Cantidad de punteros por bloque

R(a,b,c) … índice sobre a … a > 5



B+ tree unclustered

R(a,b,c):

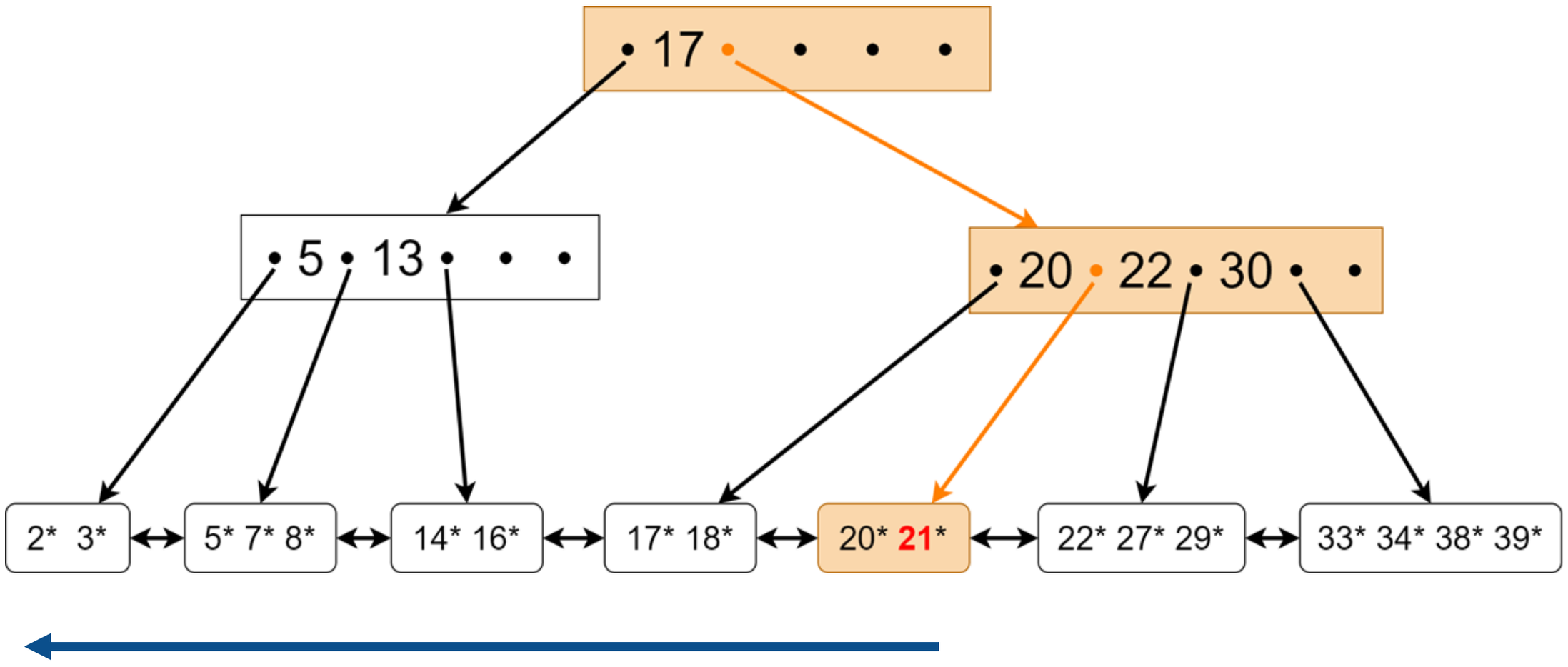
- 1M tuplas
- valor de a distribuido uniformemente en [0,1.000.000]
- B tuplas por página
- P > B punteros en una página (hojas a 100% ocupación)
- h altura del B+ tree

```
SELECT * FROM R WHERE a<21
```

Costo:

- Buscar a = 21 … h
- Scan hacia la izquierda

B+ tree unclustered



B+ tree unclustered

R(a,b,c):

- 1M tuplas
- valor de a distribuido uniformemente en [0,1.000.000]
- B tuplas por página
- P > B punteros en una página (hojas a 100% ocupación)
- h altura del B+ tree

Costo:

- Buscar a = 21 -> h
- Por uniformidad + a es llave para a = 21 -> 21 tuplas
- ¿Cuántos punteros hay en una página? -> 21/P
- Más acceder a la página en disco (unclustered) -> 21

$$h + (\left\lceil \frac{21}{P} \right\rceil - 1) + 21$$

B+ tree unclustered

R(a,b,c):

- 1M tuplas
- valor de a distribuido uniformemente en [0,1.000.000]
- B tuplas por página
- P > B punteros en una página (hojas a 100% ocupación)
- h altura del B+ tree

Costo:

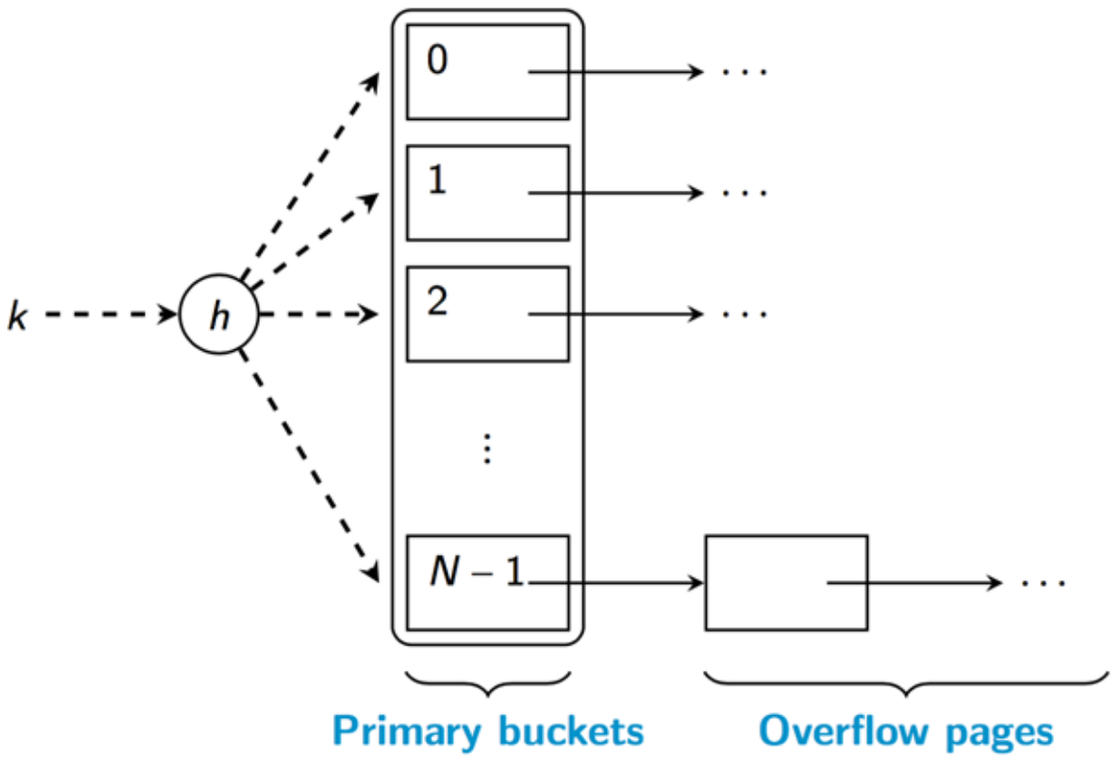
$$h + \left(\left\lceil \frac{21}{P} \right\rceil - 1\right) + 21$$

Bajar hacia a = 21

Scan a izquierda hay P punteros/página

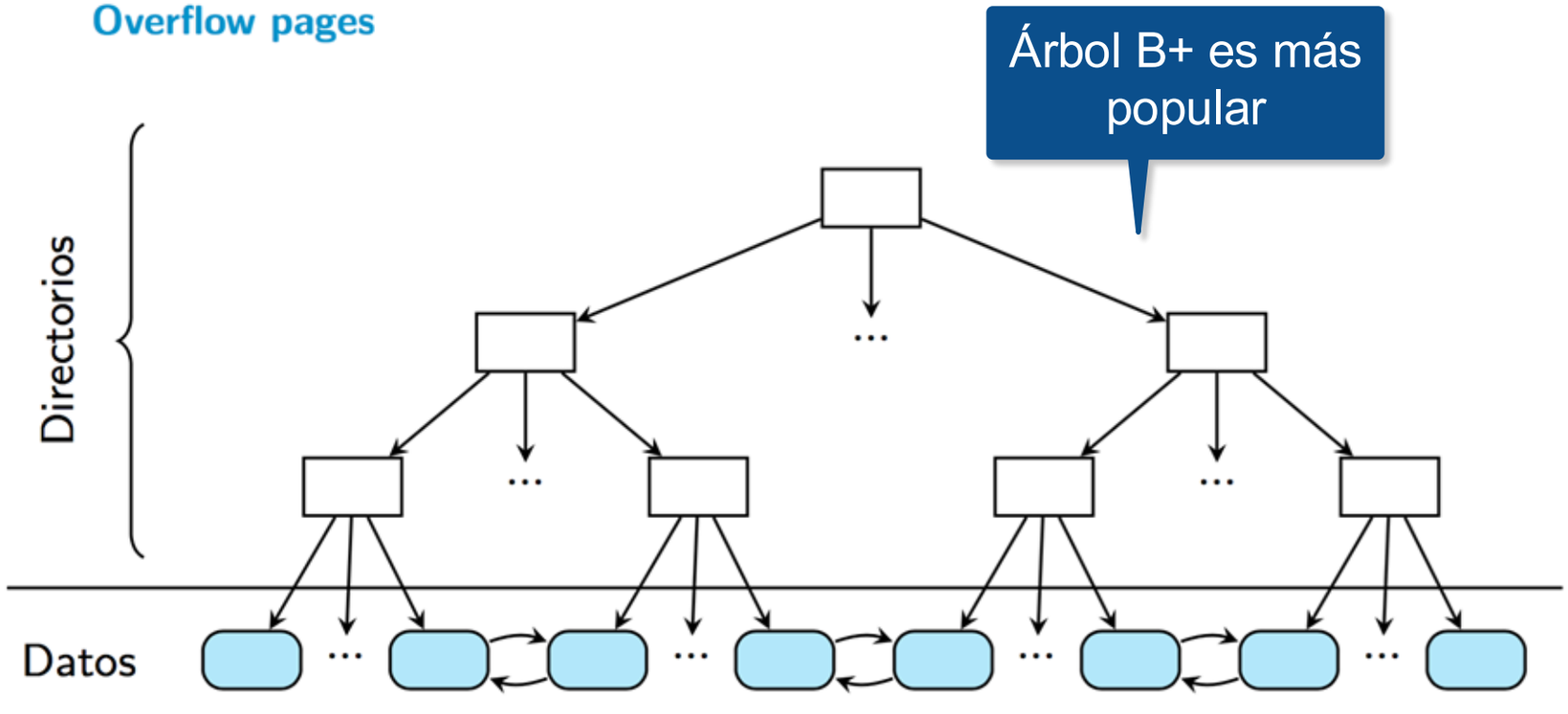
Para cada a llega un puntero, no la tupla

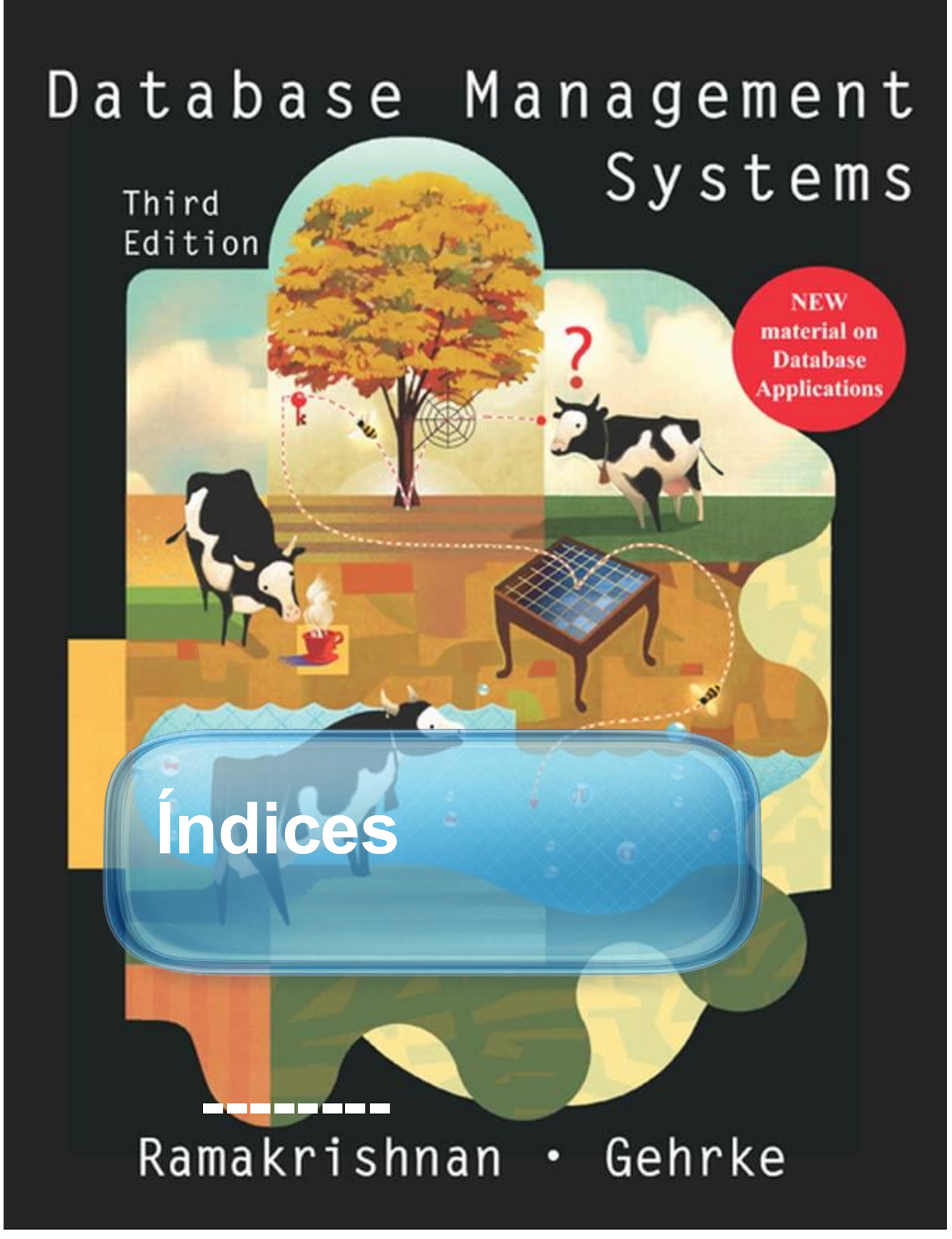
Índices: Hash vs. Árbol B+



Levemente más eficiente para búsquedas exactas asumiendo una función de hash ideal. Muy ineficientes para búsqueda de rango.
La selección de la función de hash está disponible solo en algunos DBMS

Mucho más eficiente para búsquedas por rango




Database Management Systems
Third Edition
NEW material on Database Applications
Índices
Ramakrishnan • Gehrke